
Universidad Nacional Autónoma de México

Facultad de Ciencias

Modelación y simulación computacional basada en
Agentes. 2026-1 | 7156

Proyecto Final

Bacterial Foraging Optimization (BFO)

Kevin Steve Quezada Ordoñez

stevequezada@ciencias.unam.mx

01/12/2025



Resumen

El presente proyecto estudia el algoritmo *Bacterial Foraging Optimization* (BFO) desde la perspectiva de la modelación basada en agentes. El BFO, propuesto por Kevin M. Passino en 2002, simula el comportamiento de búsqueda de alimento de la bacteria *Escherichia coli*, donde cada bacteria actúa como un agente autónomo que interactúa con su entorno y con otros agentes para resolver problemas de optimización.

El documento presenta los fundamentos biológicos del modelo, describiendo el comportamiento de cada bacteria como un agente con reglas locales de decisión. Se detalla el modelo interno del agente (quimiotaxis con patrón *run-and-tumble*), la interacción entre agentes (swarming), y los mecanismos de reproducción y dispersión que generan propiedades emergentes a nivel poblacional.

El análisis desde la perspectiva de MBA revela cómo reglas simples a nivel individual producen comportamientos colectivos complejos, logrando un balance efectivo entre exploración y explotación del espacio de búsqueda. Se presentan los parámetros globales del modelo, el diseño de la interfaz de simulación, y propuestas de visualización de la dinámica del sistema.

Índice

1. Introducción	4
1.1. Modelación Basada en Agentes y Sistemas Bio-Inspirados	4
1.2. Motivación del Estudio	4
1.3. Propuesta y Selección del Modelo	5
1.3.1. Búsqueda Bibliográfica	5
1.3.2. Análisis del Modelo Original	5
1.3.3. Lenguaje de Implementación	5
1.3.4. Análisis de Parámetros	6
1.3.5. Propiedades Emergentes	6
1.3.6. Validez del Modelo	6
1.4. Pregunta de Investigación y Objetivos	7
1.4.1. Pregunta de Investigación	7
1.4.2. Objetivo General	7
1.4.3. Objetivos Específicos	7
2. Fundamentos Biológicos	8
2.1. Biología de <i>Escherichia coli</i>	8
2.1.1. Estructura y Morfología	8
2.1.2. Sistema de Locomoción Flagelar	8
2.2. Comportamiento de Búsqueda de Alimento (Foraging)	9
2.2.1. Quimiotaxis Bacteriana	9
2.2.2. Patrones de Movimiento: Run and Tumble	9
2.2.3. Comportamiento de Enjambre (Swarming)	10
2.3. Ciclo de Vida Bacteriano Relevante	10
2.3.1. Reproducción por Fisión Binaria	10
2.3.2. Muerte Celular y Dispersión	11
3. El Algoritmo Bacterial Foraging Optimization	12
3.1. Origen y Desarrollo Histórico	12
3.2. Fundamentos del Algoritmo	12
3.2.1. Representación de Soluciones	12
3.2.2. Función de Costo (Nutrientes)	13
3.3. Parámetros del Algoritmo	13
3.4. Modelo Interno del Agente (Bacteria)	15
3.4.1. Atributos del Agente	15
3.4.2. Reglas de Comportamiento: Quimiotaxis	15
3.4.3. Interacción entre Agentes: Swarming	16
3.4.4. Mecanismos Poblacionales: Reproducción	16
3.4.5. Mecanismos Poblacionales: Eliminación y Dispersión	17
3.5. Pseudocódigo del Algoritmo	18
3.6. Diagrama de Flujo del Modelo Interno del Agente	20
3.7. Diagrama de Flujo General del Modelo	21
3.8. Diseño de Interfaz y Parámetros Globales	22
3.9. Propuesta de Visualización de la Dinámica	22

4. Análisis de la Dinámica y Propiedades Emergentes	23
4.1. Propiedades Emergentes del Sistema	23
4.2. Dinámica Local vs. Dinámica Global	23
4.2.1. Dinámica Local (Nivel Agente)	23
4.2.2. Dinámica Global (Nivel Población)	23
4.3. Complejidad Computacional	24
4.3.1. Complejidad Temporal	24
4.3.2. Complejidad Espacial	24
4.4. Ventajas del Algoritmo	24
4.5. Limitaciones del Algoritmo	25
5. Aplicaciones del BFO	26
5.1. Áreas de Aplicación	26
5.1.1. Optimización de Funciones Continuas	26
5.1.2. Ingeniería de Control	26
5.1.3. Procesamiento de Señales e Imágenes	27
5.1.4. Otras Aplicaciones	27
5.2. Ejemplo Conceptual	28
5.2.1. Ejemplo: Optimización de una Función Bidimensional	28
5.2.2. Visualización de las Fases	29
6. Resultados Experimentales	30
6.1. Experimentación Automatizada con BehaviorSpace	30
6.2. Configuración Base del Experimento	30
6.3. Escenario 1: Efecto del Tamaño de Población	31
6.4. Escenario 2: Efecto del Tamaño de Paso	33
6.5. Escenario 3: Efecto de la Probabilidad de Dispersión	35
6.6. Escenario 4: Comparación de Funciones Objetivo	36
6.7. Escenario 5: Efecto del Swarming	38
6.8. Escenario 6: Extensiones Implementadas	39
6.8.1. Obstáculos en el Entorno	39
6.8.2. Nutrientes Agotables	41
6.9. Hallazgos Importantes	42
7. Conclusiones y Reflexiones	43
7.1. Respuesta a la Pregunta de Investigación	43
7.2. Alcance del Modelo	43
7.3. Reflexiones Finales	43
Referencias	44

1. Introducción

1.1. Modelación Basada en Agentes y Sistemas Bio-Inspirados

La Modelación Basada en Agentes (MBA) es un paradigma computacional donde sistemas complejos se simulan mediante la interacción de entidades autónomas llamadas agentes, cada uno con reglas de comportamiento propias (Wilensky & Rand, 2015). Este enfoque permite estudiar cómo propiedades emergentes surgen de interacciones locales, siendo particularmente útil para modelar sistemas biológicos y sociales.

Los algoritmos bio-inspirados, que toman como modelo procesos observados en sistemas biológicos, pueden analizarse naturalmente bajo el paradigma de MBA. Entre las principales categorías se encuentran:

- **Algoritmos evolutivos:** Inspirados en la teoría de la evolución de Darwin, incluyen los algoritmos genéticos, estrategias evolutivas y programación genética (Eiben & Smith, 2015).
- **Algoritmos basados en inteligencia de enjambres:** Modelan el comportamiento colectivo de grupos de organismos, como la optimización por enjambre de partículas (PSO) y la optimización por colonia de hormigas (ACO) (Kennedy & Eberhart, 1995; Dorigo & Stützle, 2004).
- **Algoritmos basados en comportamiento microbiano:** Simulan los mecanismos de búsqueda y adaptación de microorganismos, categoría a la que pertenece el algoritmo *Bacterial Foraging Optimization* (BFO) (Passino, 2002).

El algoritmo BFO, propuesto por Kevin M. Passino en 2002, se ubica en la intersección entre los algoritmos de enjambre y los basados en comportamiento microbiano, ya que modela tanto el movimiento individual de las bacterias como su comportamiento colectivo en la búsqueda de nutrientes.

1.2. Motivación del Estudio

El algoritmo BFO es un excelente ejemplo de modelo basado en agentes por las siguientes razones:

- i.* **Agentes autónomos:** Cada bacteria actúa como un agente independiente con su propia posición, estado y reglas de comportamiento. No existe un control centralizado; el comportamiento colectivo emerge de las decisiones individuales.
- ii.* **Interacción local:** Las bacterias interactúan entre sí mediante señales químicas (swarming), ejemplificando la comunicación entre agentes que caracteriza a los MBA.
- iii.* **Propiedades emergentes:** El comportamiento global de optimización emerge de reglas simples a nivel individual (tumble, swim, reproducción), demostrando cómo sistemas complejos surgen de interacciones locales.
- iv.* **Fundamento biológico sólido:** El modelo se basa en décadas de investigación microbiológica sobre *E. coli* (Adler, 1966; Berg, 2004), proporcionando validez científica a las reglas de comportamiento de los agentes.

1.3. Propuesta y Selección del Modelo

1.3.1. Búsqueda Bibliográfica

Se realizaron búsquedas en bases de datos académicas (IEEE Xplore, Google Scholar, SpringerLink) utilizando términos como “bacterial foraging optimization”, “agent-based swarm intelligence” y “bio-inspired optimization algorithms”. Los resultados revelaron que el BFO es un algoritmo ampliamente estudiado con más de 8,000 citaciones al artículo original de Passino (2002), lo que garantiza una base teórica sólida y múltiples implementaciones de referencia.

Las fuentes principales consultadas incluyen:

- Passino, K.M. (2002). “Biomimicry of bacterial foraging for distributed optimization and control”. *IEEE Control Systems Magazine*.
- Das, S. et al. (2009). “Bacterial Foraging Optimization Algorithm: Theoretical Foundations, Analysis, and Applications”. *Foundations of Computational Intelligence*.
- Mezura-Montes, E. & Coello, C.A.C. (2011). “Modified Bacterial Foraging Optimization for Engineering Design”.

1.3.2. Análisis del Modelo Original

El modelo BFO presenta las siguientes características estructurales que lo hacen idóneo para su estudio como MBA:

- **Estados de los agentes:** Cada bacteria mantiene un estado definido por su posición en el espacio de búsqueda $\theta_i \in \mathbb{R}^p$, su valor de aptitud $J(\theta_i)$, y su historial de salud acumulado durante la quimiotaxis.
- **Reglas de evolución:** El comportamiento de cada agente sigue cuatro fases bien definidas: quimiotaxis (tumble-swim), swarming (señalización colectiva), reproducción (selección natural) y eliminación-dispersión (perturbación estocástica).
- **Inicialización:** Las bacterias se posicionan aleatoriamente en el espacio de búsqueda con distribución uniforme, sin sesgo hacia regiones particulares.
- **Condiciones de frontera:** El modelo original de Passino considera un espacio continuo sin fronteras explícitas; en la implementación se emplean condiciones periódicas (toroidales) para mantener a los agentes dentro del dominio.

1.3.3. Lenguaje de Implementación

El modelo original fue descrito mediante pseudocódigo y ecuaciones matemáticas en MATLAB. Para este proyecto, se implementó en **NetLogo 6.2.0**, una plataforma especializada en modelación basada en agentes que ofrece:

- Primitivas nativas para manejo de agentes (**turtles**) y ambiente (**patches**).
- Visualización en tiempo real de la dinámica del sistema.
- Interfaz gráfica para experimentación con parámetros.
- Herramientas de análisis como BehaviorSpace para estudios paramétricos.

1.3.4. Análisis de Parámetros

El BFO cuenta con múltiples parámetros que controlan su comportamiento. La Tabla 1.3.4 resume el efecto de variar los parámetros principales:

Parámetro	Rango típico	Efecto al modificarlo
S (población)	20–100	Mayor S aumenta exploración pero incrementa costo computacional
N_c (pasos quimiotaxis)	50–200	Más pasos permiten mayor refinamiento local
N_s (pasos swim)	2–10	Controla persistencia en direcciones favorables
$C(i)$ (tamaño paso)	0.01–0.5	Mayor paso = exploración; menor = explotación
P_{ed} (prob. dispersión)	0.1–0.3	Mayor P_{ed} evita estancamiento en óptimos locales

1.3.5. Propiedades Emergentes

1. **Autoorganización espacial:** Sin coordinación central, las bacterias tienden a agruparse en regiones de alta aptitud, formando clusters alrededor de óptimos locales y globales.
2. **Convergencia colectiva:** La combinación de reproducción selectiva y dispersión estocástica genera una dinámica poblacional que converge hacia el óptimo global.
3. **Balance exploración-explotación:** El mecanismo de tumble-swim produce naturalmente un equilibrio entre buscar nuevas regiones (tumble aleatorio) y refinar soluciones prometedoras (swim persistente).
4. **Robustez:** El sistema mantiene su capacidad de optimización incluso con perturbaciones (eliminación-dispersión), exhibiendo resiliencia ante cambios ambientales.

1.3.6. Validez del Modelo

El modelo captura los aspectos esenciales del foraging de *E. coli* documentados experimentalmente:

- ✓ El patrón *run-and-tumble* está validado por décadas de microscopía (Berg & Brown, 1972).
- ✓ La modulación de tumble según gradiente químico corresponde al mecanismo de quimiotaxis real.
- ✓ La reproducción por fisión binaria y la selección por aptitud reflejan la dinámica poblacional real.

Sin embargo, el modelo hace simplificaciones importantes:

- ✗ No modela la dinámica intracelular de señalización (cascadas de fosforilación).
- ✗ Asume un ambiente estático (aunque las extensiones implementadas añaden nutrientes dinámicos).
- ✗ Los parámetros del swarming son fenomenológicos, no derivados de mediciones reales.

El modelo no pretende predecir comportamiento bacteriano exacto, sino **abstraer los principios biológicos** que hacen efectiva la búsqueda de alimento para aplicarlos a problemas computacionales.

1.4. Pregunta de Investigación y Objetivos

1.4.1. Pregunta de Investigación

¿Cómo las reglas de comportamiento individual de las bacterias *E. coli* (quimiotaxis, swarming, reproducción y dispersión) generan propiedades emergentes de búsqueda eficiente que pueden modelarse como un sistema multiagente para resolver problemas de optimización?

1.4.2. Objetivo General

Desarrollar e implementar un modelo basado en agentes que simule el comportamiento de búsqueda de alimento de bacterias *E. coli*, analizando cómo las reglas locales de cada agente producen comportamientos emergentes útiles para la optimización.

1.4.3. Objetivos Específicos

1. Modelar el comportamiento interno de cada bacteria como un agente autónomo con reglas de quimiotaxis (patrón run-and-tumble).
2. Implementar la interacción entre agentes mediante el mecanismo de swarming (atracción-repulsión).
3. Analizar las propiedades emergentes del sistema: autoorganización hacia óptimos y balance exploración-explotación.
4. Visualizar la dinámica del sistema mediante series de tiempo, distribución espacial y métricas de convergencia.

2. Fundamentos Biológicos

Para comprender el algoritmo *Bacterial Foraging Optimization*, es fundamental conocer los principios biológicos que lo inspiran. Esta sección presenta los fundamentos del comportamiento de la bacteria *Escherichia coli*, cuyas estrategias de búsqueda de alimento constituyen la base del algoritmo BFO.

2.1. Biología de *Escherichia coli*

Escherichia coli es una bacteria gram-negativa (no se tiñen de azul oscuro o de violeta por la tinción de Gram) perteneciente a la familia *Enterobacteriaceae*. Descubierta por el pediatra alemán Theodor Escherich en 1885, esta bacteria se ha convertido en uno de los organismos modelo más estudiados en biología molecular y microbiología (Blount, 2015).

E. coli habita principalmente en el tracto intestinal de mamíferos, incluyendo humanos, donde forma parte de la flora intestinal normal. Sin embargo, también puede sobrevivir en ambientes externos como el suelo y el agua. Su capacidad para crecer rápidamente en condiciones de laboratorio y su genoma relativamente simple la han convertido en un organismo fundamental para la investigación (Madigan et al., 2019).

2.1.1. Estructura y Morfología

E. coli presenta una morfología de bacilo (forma de bastón) con dimensiones típicas de aproximadamente $2\text{ }\mu\text{m}$ de longitud y $0.5\text{--}1\text{ }\mu\text{m}$ de diámetro (Neidhardt et al., 1990). Las estructuras más relevantes para comprender su comportamiento de búsqueda de alimento son:

- **Membrana celular y pared:** La *E. coli* posee una estructura de doble membrana con una capa de peptidoglicano entre ellas. La membrana externa contiene receptores que detectan señales químicas.
- **Flagelos:** Cada célula de *E. coli* posee entre 4 y 10 flagelos distribuidos alrededor de su cuerpo (disposición peritrica). Cada flagelo es un filamento helicoidal de aproximadamente 20 nm de diámetro y hasta $15\text{ }\mu\text{m}$ de longitud, compuesto principalmente por la proteína flagelina (Berg, 2003).
- **Quimiorreceptores:** La bacteria posee proteínas receptoras transmembranales llamadas *methyl-accepting chemotaxis proteins* (MCPs), localizadas principalmente en los polos celulares. Estos receptores detectan la presencia de sustancias químicas atrayentes (como aminoácidos y azúcares) y repelentes (como ácidos y alcoholes) en el ambiente (Wadhams & Armitage, 2004).
- **Motor flagelar:** Cada flagelo está conectado a un motor rotatorio molecular en la membrana celular. Este motor de 45 nm de diámetro, puede rotar a velocidades de hasta 300 revoluciones por segundo y es impulsado por el flujo de protones a través de la membrana (Berg, 2003).

2.1.2. Sistema de Locomoción Flagelar

Este sistema representa una de las máquinas moleculares más estudiadas en biología. El movimiento bacteriano depende de la rotación coordinada de sus flagelos. La velocidad típica de natación es de aproximadamente $20\text{--}30\text{ }\mu\text{ m/s}$, lo que equivale a recorrer entre 10 y 20 veces su longitud por segundo.

- **Rotación antihoraria (CCW):** Cuando todos los motores rotan en sentido antihorario, los flagelos se agrupan formando un haz helicoidal que propulsa a la bacteria hacia adelante en una trayectoria relativamente recta. Este estado se denomina *run* (Turner et al., 2000).
- **Rotación horaria (CW):** Cuando uno o más motores cambian a rotación horaria, el haz flagelar se desorganiza, causando que la bacteria gire de manera errática sin desplazamiento significativo. Este estado se denomina *tumble* y resulta en una reorientación de la dirección (Berg, 2004).

2.2. Comportamiento de Búsqueda de Alimento (Foraging)

El *foraging* o comportamiento de búsqueda de alimento es una estrategia fundamental para la supervivencia de los organismos. En el contexto bacteriano, este comportamiento implica la capacidad de localizar y moverse hacia fuentes de nutrientes mientras se evitan sustancias nocivas.

Desde una perspectiva evolutiva, las estrategias de foraging pueden interpretarse como soluciones optimizadas por la selección natural para maximizar la adquisición de energía minimizando los costos asociados a la búsqueda (Stephens & Krebs, 1986). Las bacterias que desarrollan estrategias más eficientes de búsqueda de alimento tienen mayor probabilidad de sobrevivir y reproducirse, transmitiendo estas capacidades a su descendencia.

2.2.1. Quimiotaxis Bacteriana

La quimiotaxis es el movimiento direccional de un organismo en respuesta a una concentración química. En bacterias, este fenómeno fue estudiado sistemáticamente por primera vez por Julius Adler en la década de 1960, estableciendo las bases para la comprensión moderna del comportamiento bacteriano.

- **Quimiotaxis positiva:** Movimiento hacia mayores concentraciones de sustancias atrayentes, como aminoácidos (aspartato, serina), azúcares (glucosa, galactosa, ribosa) y dipéptidos.
- **Quimiotaxis negativa:** Movimiento alejándose de sustancias repelentes, como ácidos grasos, alcoholes, y ciertas sales metálicas.

Un aspecto fundamental de la quimiotaxis bacteriana es la **adaptación sensorial**. Las bacterias no detectan concentraciones absolutas, sino cambios temporales en la concentración química. Después de detectar un cambio, el sistema sensorial se adapta gradualmente al nuevo nivel de fondo, permitiendo a la bacteria responder a cambios subsecuentes. Esta adaptación, mediada por la metilación de los receptores MCP, permite a *E. coli* responder a un amplio rango de concentraciones químicas (Berg & Brown, 1972).

2.2.2. Patrones de Movimiento: Run and Tumble

- **Run (Natación):** Durante un *run*, la bacteria se desplaza en línea aproximadamente recta a velocidad constante. La duración promedio de un *run* es de 1 segundo cuando la bacteria se encuentra en un ambiente químicamente homogéneo, aunque puede variar entre 0.1 y varios segundos.
- **Tumble (Volteo):** Durante un *tumble*, que dura típicamente 0.1 segundos, la bacteria cambia de orientación de manera esencialmente aleatoria. La nueva dirección de movimiento tiene poca o ninguna correlación con la dirección anterior.

La clave de la quimiotaxis efectiva radica en la **modulación de la frecuencia de tumbles**. Cuando la bacteria nada en una dirección favorable (hacia mayor concentración de atrayente o menor concentración de repelente), la frecuencia de tumbles disminuye, prolongando los *runs* en esa dirección. Cuando nada en una dirección desfavorable, la frecuencia de tumbles aumenta, causando reorientaciones más frecuentes.

Este mecanismo, conocido como *biased random walk* (caminata aleatoria sesgada), permite a la bacteria navegar eficientemente hacia condiciones favorables sin necesidad de detectar directamente la dirección del químico. El sesgo estadístico acumulado sobre múltiples ciclos de run-and-tumble resulta en un movimiento neto hacia el atrayente.

2.2.3. Comportamiento de Enjambre (Swarming)

Además de su comportamiento individual, *E. coli* y otras bacterias exhiben comportamientos colectivos cuando se encuentran en poblaciones densas. El *swarming* bacteriano es un tipo de movilidad social caracterizado por el movimiento coordinado de grupos de células (Kearns, 2010).

El *swarming* difiere del movimiento natatorio individual en varios aspectos:

- **Diferenciación celular:** Las células que participan en *swarming* frecuentemente presentan cambios morfológicos, incluyendo elongación y aumento en el número de flagelos.
- **Movimiento coordinado:** Las células se mueven en grupos o *rafts*, con comportamiento colectivo que no se observa en células individuales.
- **Señalización química:** La comunicación célula-célula juega un papel importante en la coordinación del grupo.

Un mecanismo relevante para el comportamiento colectivo es el *quorum sensing*, un sistema de comunicación bacteriana basado en la producción y detección de moléculas señalizadoras llamadas autoinductores. Cuando la densidad poblacional alcanza un umbral crítico, la concentración de autoinductores activa cambios en la expresión génica que coordinan comportamientos poblacionales (Miller & Bassler, 2001).

En el contexto del algoritmo BFO, el comportamiento de *swarming* se modela como una interacción atractiva-repulsiva entre bacterias virtuales, donde las células se atraen mutuamente a distancias intermedias pero se repelen cuando están muy cercanas, evitando la agregación excesiva y manteniendo la diversidad de búsqueda (Passino, 2002).

2.3. Ciclo de Vida Bacteriano Relevante

El ciclo de vida de *E. coli* incluye procesos de reproducción y muerte que son modelados en el algoritmo BFO como mecanismos de intensificación y diversificación de la búsqueda.

2.3.1. Reproducción por Fisión Binaria

E. coli se reproduce asexualmente mediante fisión binaria, un proceso en el cual una célula madre se divide en dos células hijas genéticamente idénticas (Madigan et al., 2019).

- i. **Replicación del ADN:** El cromosoma circular bacteriano se duplica comenzando desde un origen de replicación único.
- ii. **Elongación celular:** La célula aumenta su tamaño aproximadamente al doble de su longitud.
- iii. **Formación del septo:** Se forma una pared divisoria (septo) en el centro de la célula.
- iv. **División:** La célula se divide en dos células hijas, cada una con una copia del cromosoma.

En condiciones óptimas de laboratorio (medio rico, 37°C), el tiempo de generación de *E. coli* es de aproximadamente 20 minutos. Sin embargo, en ambientes naturales con disponibilidad limitada de nutrientes, el tiempo de generación puede extenderse considerablemente (Neidhardt et al., 1990).

Un aspecto importante es que las bacterias que han tenido acceso a más nutrientes tienden a dividirse más rápidamente que aquellas en condiciones nutricionales pobres. Este principio se modela en el algoritmo BFO mediante la reproducción preferencial de las bacterias con mejores valores de función de costo.

2.3.2. Muerte Celular y Dispersión

- **Factores ambientales adversos:** Cambios en pH, temperatura, disponibilidad de oxígeno o nutrientes pueden resultar en muerte celular. Las bacterias en regiones con condiciones desfavorables tienen menor probabilidad de supervivencia.
- **Predación y competencia:** Bacteriófagos, protozoos y otras bacterias pueden eliminar células de la población.
- **Eventos catastróficos:** Perturbaciones ambientales pueden eliminar porciones significativas de una población, forzando a los sobrevivientes a colonizar nuevos territorios.
- **Dispersión activa:** Algunas bacterias pueden migrar activamente a nuevos ambientes en busca de mejores condiciones, especialmente cuando los recursos locales se agotan.

En BFO, estos fenómenos se modelan mediante el operador de **eliminación-dispersión**, donde algunas bacterias son eliminadas con cierta probabilidad y reubicadas aleatoriamente en el espacio de búsqueda. Este mecanismo cumple una función crucial: prevenir el estancamiento del algoritmo en óptimos locales al introducir diversidad en la población y explorar nuevas regiones del espacio de soluciones (Passino, 2002).

3. El Algoritmo Bacterial Foraging Optimization

Esta sección presenta el algoritmo *Bacterial Foraging Optimization* (BFO), describiendo su origen histórico, los fundamentos que lo constituyen y los parámetros que controlan su comportamiento. La descripción detallada de las fases del algoritmo se desarrolla estableciendo la correspondencia con los fenómenos biológicos presentados en la sección anterior.

3.1. Origen y Desarrollo Histórico

El algoritmo *Bacterial Foraging Optimization* fue propuesto por Kevin M. Passino, profesor de ingeniería eléctrica y de computación en *The Ohio State University*, en su artículo seminal *Biomimicry of bacterial foraging for distributed optimization and control* publicado en *IEEE Control Systems Magazine* en junio de 2002 (Passino, 2002).

El desarrollo del BFO surgió en el contexto de la investigación en sistemas de control distribuido. Passino observó que el comportamiento de foraging de las bacterias, perfeccionado a lo largo de millones de años de evolución, representaba una estrategia de búsqueda altamente eficiente que podría traducirse en un algoritmo de optimización. La motivación original era desarrollar métodos de optimización robustos para aplicaciones en ingeniería de control, donde se requieren soluciones que funcionen bien bajo incertidumbre y perturbaciones (Passino, 2002).

Desde su publicación inicial, el BFO ha generado considerable interés en la comunidad científica:

- **2002–2005:** Período de introducción y primeras aplicaciones. Los investigadores comenzaron a explorar el algoritmo en diversos problemas de optimización y control, validando su efectividad en funciones de prueba estándar.
- **2005–2009:** Período de maduración. Se propusieron múltiples variantes y mejoras, incluyendo versiones adaptativas y algoritmos híbridos que combinan BFO con otras metaheurísticas como PSO y algoritmos genéticos (Kim et al., 2007).
- **2009–presente:** El algoritmo se ha consolidado como una metaheurística establecida, con aplicaciones en áreas que van desde la ingeniería de potencia hasta el procesamiento de imágenes y la bioinformática.

3.2. Fundamentos del Algoritmo

El BFO es un algoritmo de optimización basado en población que simula el comportamiento de búsqueda de alimento de las bacterias *E. coli*.

3.2.1. Representación de Soluciones

En el algoritmo BFO, cada bacteria virtual representa una solución candidata al problema de optimización. La posición de una bacteria en el espacio de búsqueda corresponde a un vector de variables de decisión:

$$\theta^i = [\theta_1^i, \theta_2^i, \dots, \theta_p^i]^T \in \mathbb{R}^p$$

donde θ^i representa la posición de la i -ésima bacteria, p es la dimensión del problema (número de variables de decisión), y cada componente θ_m^i corresponde al valor de la m -ésima variable.

La población del algoritmo consiste en S bacterias, formando un conjunto de soluciones candidatas que evolucionan simultáneamente:

$$P = \{\theta^1, \theta^2, \dots, \theta^S\}$$

La inicialización de la población se realiza típicamente de manera aleatoria, distribuyendo las bacterias uniformemente dentro de los límites del espacio de búsqueda. Si el dominio de cada variable está definido por $[\theta_m^{\min}, \theta_m^{\max}]$, la posición inicial de cada bacteria se genera como:

$$\theta_m^i = \theta_m^{\min} + rand() \cdot (\theta_m^{\max} - \theta_m^{\min})$$

donde $rand()$ genera un número aleatorio uniformemente distribuido en $[0, 1]$.

3.2.2. Función de Costo (Nutrientes)

La función de costo (o función objetivo) en el BFO representa la concentración de nutrientes en el ambiente virtual. Siguiendo la analogía biológica:

- **Alta concentración de nutrientes** corresponde a regiones del espacio de búsqueda con buenos valores de la función objetivo (soluciones de alta calidad).
- **Baja concentración de nutrientes** corresponde a regiones con malos valores de la función objetivo (soluciones de baja calidad).
- **Sustancias nocivas** pueden modelarse como regiones donde la función de costo tiene valores extremadamente desfavorables.

Para un problema de minimización, la función de costo $J(\theta)$ se define de manera que valores menores indiquen mejores soluciones. Las bacterias buscan moverse hacia regiones de menor costo, análogo a cómo las bacterias reales buscan regiones de mayor concentración de nutrientes.

En caso de problemas de maximización, la función puede transformarse multiplicando por -1 :

$$J_{\min}(\theta) = -f(\theta) \quad \text{o} \quad J_{\min}(\theta) = \frac{1}{1 + f(\theta)}$$

donde $f(\theta)$ es la función original a maximizar.

3.3. Parámetros del Algoritmo

Símbolo	Nombre	Descripción
S	Población	Número de bacterias en la población
p	Dimensión	Dimensión del espacio de búsqueda
N_c	Pasos quimiotácticos	Número de iteraciones de quimiotaxis
N_s	Longitud de nado	Máximo de pasos en una dirección
N_{re}	Pasos de reproducción	Número de ciclos reproductivos
N_{ed}	Eliminación-dispersión	Número de eventos de eliminación
P_{ed}	Probabilidad	Probabilidad de eliminación-dispersión
$C(i)$	Tamaño de paso	Tamaño del paso quimiotáctico

A continuación se describe cada parámetro en detalle:

- **Tamaño de población (S):** Determina el número de bacterias (soluciones candidatas) que evolucionan simultáneamente. Un valor mayor proporciona mejor exploración pero aumenta el costo computacional. Valores típicos oscilan entre 20 y 100 bacterias (Das et al., 2009).
- **Dimensión (p):** Corresponde al número de variables del problema de optimización. Este parámetro está determinado por el problema específico a resolver.
- **Número de pasos quimiotácticos (N_c):** Define cuántas iteraciones de quimiotaxis (movimientos de tumble y swim) se realizan antes de cada evento de reproducción. Controla la intensidad de la búsqueda local. Valores típicos: 50–200 (Passino, 2002).
- **Longitud de nado (N_s):** Especifica el número máximo de pasos consecutivos que una bacteria puede dar en la misma dirección mientras mejora. Valores mayores favorecen la explotación de direcciones prometedoras. Valores típicos: 4–10 (Das et al., 2009).
- **Número de pasos de reproducción (N_{re}):** Indica cuántos ciclos de reproducción ocurren antes de cada evento de eliminación-dispersión. Valores típicos: 4–10 (Passino, 2002).
- **Número de eventos de eliminación-dispersión (N_{ed}):** Controla cuántas veces se aplica el operador de eliminación-dispersión durante la ejecución del algoritmo. Valores típicos: 2–5 (Das et al., 2009).
- **Probabilidad de eliminación-dispersión (P_{ed}):** Determina la probabilidad de que cada bacteria sea eliminada y reubicada aleatoriamente. Valores bajos mantienen más continuidad en la búsqueda, mientras que valores altos aumentan la exploración. Valores típicos: 0.1–0.25 (Passino, 2002).
- **Tamaño del paso quimiotáctico ($C(i)$):** Define la magnitud del movimiento durante cada paso de quimiotaxis. Puede ser constante para todas las bacterias o adaptativo. Un valor típico es $C(i) = 0,1$ para problemas normalizados (Das et al., 2009).

La selección adecuada de estos parámetros depende del problema específico y frecuentemente requiere experimentación. El número total de evaluaciones de la función objetivo en una ejecución completa del BFO está dado aproximadamente por:

$$\text{Evaluaciones} \approx S \times N_c \times N_{re} \times N_{ed}$$

Esta fórmula permite estimar el costo computacional del algoritmo y ajustar los parámetros según los recursos disponibles.

3.4. Modelo Interno del Agente (Bacteria)

Desde la perspectiva de MBA, cada bacteria es un agente autónomo. A continuación se describe el modelo interno que gobierna el comportamiento de cada agente en cada tick de simulación.

3.4.1. Atributos del Agente

Cada bacteria-agente posee los siguientes atributos internos:

- **Posición** $\theta^i \in \mathbb{R}^p$: Vector que representa la ubicación del agente en el espacio de búsqueda.
- **Dirección actual** $\Delta(i)$: Vector unitario que indica la dirección de movimiento.
- **Costo actual** J^i : Valor de la función objetivo en la posición actual.
- **Salud acumulada** J_{health}^i : Suma de costos a lo largo de los pasos quimiotácticos.
- **Contador de nado** m : Número de pasos consecutivos en la misma dirección.

3.4.2. Reglas de Comportamiento: Quimiotaxis

La quimiotaxis es el comportamiento principal que ejecuta cada agente en cada tick. Implementa el patrón de movimiento *run-and-tumble*:

Operación Tumble (Volteo): Cada bacteria genera una dirección de movimiento aleatoria. El vector de dirección $\Delta(i) \in \mathbb{R}^p$ se genera con componentes aleatorias en el intervalo $[-1, 1]$. La nueva posición de la bacteria se calcula como:

$$\theta^i(j+1, k, l) = \theta^i(j, k, l) + C(i) \cdot \frac{\Delta(i)}{\sqrt{\Delta^T(i)\Delta(i)}}$$

- $\theta^i(j, k, l)$ es la posición de la bacteria i en el paso quimiotáctico j , ciclo de reproducción k , y evento de eliminación-dispersión l
- $C(i)$ es el tamaño del paso quimiotáctico
- El término $\frac{\Delta(i)}{\sqrt{\Delta^T(i)\Delta(i)}}$ normaliza el vector de dirección a magnitud unitaria

Operación Swim (Nado): Después de un tumble, si la nueva posición presenta un mejor valor de función de costo (menor J), la bacteria continúa moviéndose en la misma dirección. Este proceso se repite hasta que:

1. El agente alcanza el límite máximo de pasos de nado (N_s)
2. El movimiento ya no produce mejora en la función de costo

Este mecanismo permite a los agentes explotar direcciones prometedoras, similar a cómo las bacterias reales prolongan sus *runs* cuando nadan hacia concentraciones favorables de nutrientes.

3.4.3. Interacción entre Agentes: Swarming

El swarming modela la comunicación entre agentes. Cada bacteria percibe a las demás mediante una función de interacción que modifica su percepción del entorno, agregando términos de atracción y repulsión.

La función de costo efectiva J_{cc} que incluye la interacción célula-célula se define como:

$$J_{cc}(\theta, P(j, k, l)) = \sum_{i=1}^S J_{cc}^i(\theta, \theta^i(j, k, l))$$

donde cada término J_{cc}^i representa la contribución de la bacteria i a la señal colectiva:

$$J_{cc}^i(\theta, \theta^i) = -d_{attract} \exp\left(-w_{attract} \sum_{m=1}^p (\theta_m - \theta_m^i)^2\right) + h_{repellant} \exp\left(-w_{repellant} \sum_{m=1}^p (\theta_m - \theta_m^i)^2\right)$$

Los parámetros de esta ecuación controlan el comportamiento de enjambre:

- $d_{attract}$: profundidad del atrayente (magnitud de la atracción)
- $w_{attract}$: ancho del atrayente (alcance de la atracción)
- $h_{repellant}$: altura del repelente (magnitud de la repulsión)
- $w_{repellant}$: ancho del repelente (alcance de la repulsión)

El efecto neto es que los agentes se atraen mutuamente a distancias intermedias (favoreciendo la agregación) pero se repelen a distancias muy cortas (evitando la concentración y manteniendo diversidad).

3.4.4. Mecanismos Poblacionales: Reproducción

La reproducción modela la selección natural a nivel poblacional. Los agentes más exitosos (menor costo acumulado) se duplican, mientras que los menos exitosos son eliminados.

El proceso de reproducción se ejecuta de la siguiente manera:

1. **Cálculo de salud:** Para cada bacteria i , se calcula un valor de “salud” J_{health}^i como la suma acumulada de los valores de función de costo obtenidos durante todos los pasos quimiotácticos:

$$J_{health}^i = \sum_{j=1}^{N_c} J(i, j, k, l)$$

Las bacterias con menor J_{health}^i (en problemas de minimización) son consideradas más “saludables”, ya que han encontrado consistentemente mejores regiones del espacio de búsqueda.

2. **Ordenamiento:** Las S bacterias se ordenan de manera ascendente según su valor de salud.
3. **Selección y reproducción:** Las $S_r = S/2$ bacterias menos saludables son eliminadas. Las S_r bacterias más saludables se dividen, creando cada una una copia ubicada en la misma posición.
4. **Conservación del tamaño:** Después de la reproducción, la población mantiene el tamaño S .

Este mecanismo es análogo a la selección elitista y asegura que los agentes exitosos tengan mayor influencia en las generaciones posteriores.

3.4.5. Mecanismos Poblacionales: Eliminación y Dispersión

La eliminación-dispersión modela eventos estocásticos que afectan a las poblaciones, como cambios drásticos en el ambiente o migración hacia nuevos hábitats.

El propósito principal es introducir diversidad en la población de agentes y prevenir la convergencia prematura hacia óptimos locales.

1. Para cada agente i en la población:

- a) Generar un número aleatorio $r \in [0, 1]$
- b) Si $r < P_{ed}$:
 - Eliminar el agente de su posición actual
 - Reubicarlo en una posición aleatoria dentro del espacio de búsqueda

La probabilidad P_{ed} controla el balance entre continuidad y exploración:

- Valores bajos de P_{ed} (ej. 0.1) mantienen mayor continuidad en la búsqueda, preservando el conocimiento acumulado.
- Valores altos de P_{ed} (ej. 0.25) aumentan la exploración, pero pueden descartar soluciones buenas.

Este operador es particularmente importante para problemas multimodales, donde múltiples óptimos locales pueden atrapar al algoritmo. La dispersión aleatoria permite descubrir nuevas regiones del espacio de búsqueda que de otra manera serían inaccesibles.

3.5. Pseudocódigo del Algoritmo

Los Algoritmos 1 y 2 presentan el pseudocódigo completo del BFO, dividido en dos partes. La primera parte muestra la inicialización y la fase de quimiotaxis, mientras que la segunda presenta las fases de reproducción y eliminación-dispersión.

Algorithm 1 Bacterial Foraging Optimization – Parte 1: Inicialización y Quimiotaxis

Require: Parámetros: $S, p, N_c, N_s, N_{re}, N_{ed}, P_{ed}, C(i)$

Require: Función de costo $J(\theta)$, límites del espacio de búsqueda

Ensure: Mejor solución encontrada θ_{best}

```

1: Inicialización:
2: for  $i = 1$  to  $S$  do
3:   Inicializar  $\theta^i$  aleatoriamente en el espacio de búsqueda
4:    $J_{health}^i \leftarrow 0$ 
5: end for
6:
7: for  $l = 1$  to  $N_{ed}$  do                                     ▷ Ciclo de Eliminación-Dispersión
8:   for  $k = 1$  to  $N_{re}$  do                                       ▷ Ciclo de Reproducción
9:     for  $j = 1$  to  $N_c$  do                                       ▷ Ciclo de Quimiotaxis
10:      for  $i = 1$  to  $S$  do                                       ▷ Para cada bacteria
11:
12:        // Fase de Quimiotaxis con Swarming opcional
13:         $J(i, j, k, l) \leftarrow J(\theta^i) + J_{cc}(\theta^i, P)$                                      ▷ Evaluar costo
14:         $J_{last} \leftarrow J(i, j, k, l)$ 
15:
16:        // Tumble: generar dirección aleatoria
17:        Generar  $\Delta(i)$  con componentes aleatorias en  $[-1, 1]$ 
18:         $\theta^i \leftarrow \theta^i + C(i) \cdot \frac{\Delta(i)}{\|\Delta(i)\|}$ 
19:
20:        // Swim: continuar si hay mejora
21:         $m \leftarrow 0$ 
22:        while  $m < N_s$  do
23:           $J_{new} \leftarrow J(\theta^i) + J_{cc}(\theta^i, P)$ 
24:          if  $J_{new} < J_{last}$  then
25:             $J_{last} \leftarrow J_{new}$ 
26:             $\theta^i \leftarrow \theta^i + C(i) \cdot \frac{\Delta(i)}{\|\Delta(i)\|}$ 
27:             $m \leftarrow m + 1$ 
28:          else
29:             $m \leftarrow N_s$                                      ▷ Terminar nado
30:          end if
31:        end while
32:
33:         $J_{health}^i \leftarrow J_{health}^i + J_{last}$                                      ▷ Acumular salud
34:      end for
35:    end for
36:    Continúa en Algoritmo 2...
37:  end for
38: end for

```

Algorithm 2 Bacterial Foraging Optimization – Parte 2: Reproducción y Eliminación-Dispersión

```

45: for  $l = 1$  to  $N_{ed}$  do                                ▷ Continua ciclo Eliminación-Dispersión
46:   for  $k = 1$  to  $N_{re}$  do                                    ▷ Continua ciclo Reproducción
47:     // Fase de Reproducción (después de quimiotaxis)
48:     Ordenar bacterias por  $J_{health}^i$  (ascendente)
49:     Las  $S/2$  bacterias con mayor  $J_{health}$  mueren
50:     Las  $S/2$  bacterias con menor  $J_{health}$  se dividen
51:     Reiniciar  $J_{health}^i \leftarrow 0$  para todas las bacterias
52:   end for                                                ▷ Fin ciclo Reproducción
53:
54:   // Fase de Eliminación-Dispersión
55:   for  $i = 1$  to  $S$  do
56:     if  $rand() < P_{ed}$  then
57:       Reubicar  $\theta^i$  aleatoriamente en el espacio de búsqueda
58:     end if
59:   end for
60: end for                                                ▷ Fin ciclo Eliminación-Dispersión
61:
62:  $\theta_{best} \leftarrow$  bacteria con menor valor de  $J(\theta)$ 
63: return  $\theta_{best}$ 

```

3.6. Diagrama de Flujo del Modelo Interno del Agente

El siguiente diagrama muestra el proceso de decisión que ejecuta cada bacteria-agente en cada tick de simulación (fase de quimiotaxis):

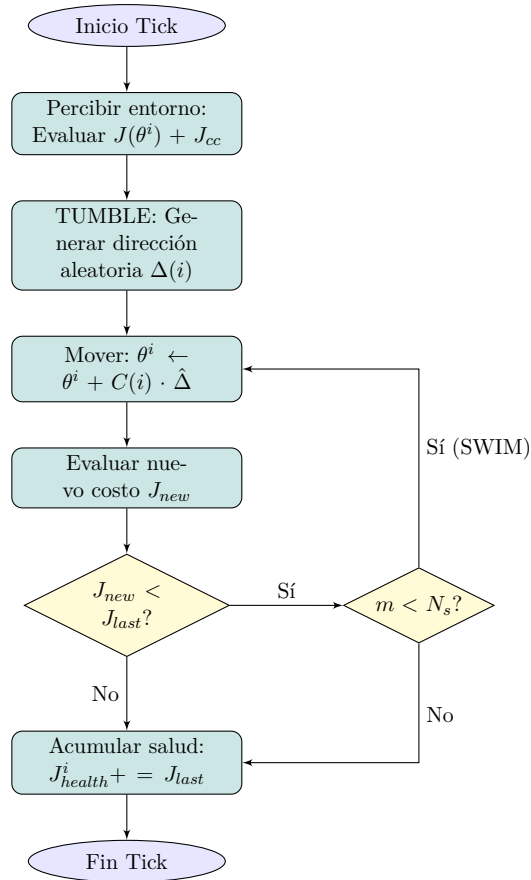


Figura 1: Diagrama de flujo del modelo interno del agente (bacteria) por tick

Explicación de cada paso:

1. **Percibir entorno:** El agente evalúa la función de costo en su posición actual, incluyendo la señal de swarming de otros agentes.
2. **TUMBLE:** Genera una dirección de movimiento aleatoria (exploración).
3. **Mover:** Actualiza su posición moviéndose en la dirección generada.
4. **Evaluar:** Mide el nuevo costo para decidir si la dirección es favorable.
5. **Decisión SWIM:** Si hay mejora y no se ha alcanzado N_s , continúa en la misma dirección.
6. **Acumular salud:** Registra el costo para futuras decisiones de reproducción.

3.7. Diagrama de Flujo General del Modelo

El siguiente diagrama muestra el flujo general del modelo considerando todos los procesos del entorno y la población de agentes:

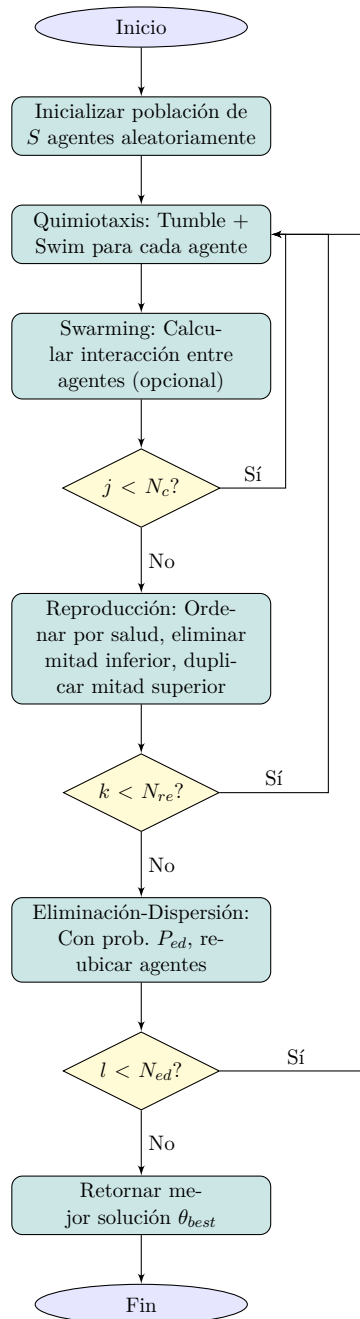


Figura 2: Diagrama de flujo general del modelo BFO

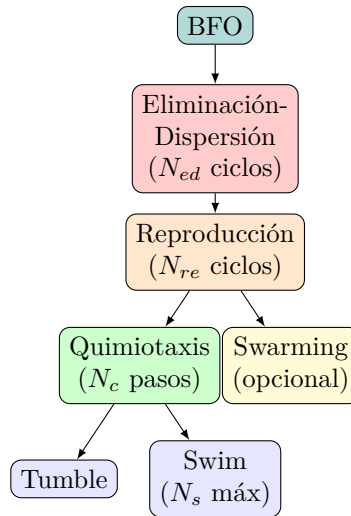


Figura 3: Estructura jerárquica de las fases del algoritmo BFO

3.8. Diseño de Interfaz y Parámetros Globales

Parámetro	Tipo Control	Rango	Descripción
S (población)	Slider	10–100	Número de agentes-bacteria
N_c (quimiotaxis)	Slider	10–100	Pasos por ciclo quimiotáctico
N_s (swim máx)	Slider	2–10	Máximo pasos en una dirección
N_{re} (reproducción)	Slider	1–10	Ciclos de reproducción
N_{ed} (elim-disp)	Slider	1–5	Ciclos de eliminación-dispersión
P_{ed} (prob. disp)	Slider	0.0–0.5	Probabilidad de dispersión
C (paso)	Slider	0.01–0.5	Tamaño del paso quimiotáctico
Swarming	Switch	On/Off	Activar interacción entre agentes
Función objetivo	Chooser	Lista	Seleccionar función de prueba

3.9. Propuesta de Visualización de la Dinámica

- Vista espacial 2D:** Mostrar la distribución de agentes en el espacio de búsqueda, coloreados según su valor de fitness. Esto permite observar la autoorganización hacia óptimos.
- Serie temporal del mejor fitness:** Gráfica que muestre la evolución del mejor valor encontrado a lo largo de los ticks, permitiendo observar la velocidad de convergencia.
- Serie temporal del fitness promedio:** Muestra la calidad promedio de la población.
- Histograma de distribución de fitness:** Visualiza la diversidad de la población en términos de calidad de soluciones.
- Contador de agentes por fase:** Monitor que muestre cuántos agentes están en swim vs. tumble.
- Densidad espacial:** Mapa de calor que muestre las regiones más pobladas.

4. Análisis de la Dinámica y Propiedades Emergentes

4.1. Propiedades Emergentes del Sistema

Desde la perspectiva de MBA, el BFO exhibe varias propiedades emergentes que surgen de las interacciones locales entre agentes:

- **Autoorganización hacia óptimos:** Sin coordinación centralizada, la población de agentes tiende a concentrarse en regiones de alto fitness. Esta autoorganización emerge de las reglas simples de quimiotaxis individual.
- **Balance exploración-explotación emergente:** El comportamiento colectivo logra un equilibrio entre búsqueda global y local:
 - El *tumble* y la eliminación-dispersión favorecen la exploración.
 - El *swim* y la reproducción favorecen la explotación.
- **Formación de clusters:** Con el swarming activado, los agentes tienden a formar grupos en regiones prometedoras, similar al comportamiento de colonias bacterianas reales.
- **Robustez poblacional:** El sistema mantiene diversidad a través de múltiples mecanismos, evitando el colapso hacia un único punto.

4.2. Dinámica Local vs. Dinámica Global

4.2.1. Dinámica Local (Nivel Agente)

A nivel individual, cada agente ejecuta un ciclo de percepción-decisión-acción:

1. Percibe el gradiente local del entorno (y señales de otros agentes si hay swarming)
2. Decide entre explorar (*tumble*) o explotar (*swim*) basándose en cambios de fitness
3. Actualiza su posición y acumula información de salud

4.2.2. Dinámica Global (Nivel Población)

A nivel poblacional observamos:

1. **Fase temprana:** Dispersión uniforme, alta exploración
2. **Fase intermedia:** Formación de clusters cerca de óptimos locales
3. **Fase tardía:** Concentración en óptimos globales (si se encontraron)

4.3. Complejidad Computacional

4.3.1. Complejidad Temporal

El costo computacional del BFO está dominado por el número de evaluaciones de la función objetivo. Considerando la estructura de ciclos anidados del algoritmo, el número de evaluaciones en el peor caso:

$$E_{total} = S \times N_c \times (1 + N_s) \times N_{re} \times N_{ed}$$

donde el factor $(1 + N_s)$ considera una evaluación inicial más hasta N_s evaluaciones adicionales en el swim. En términos de notación asintótica, la complejidad temporal es:

$$O(S \cdot N_c \cdot N_s \cdot N_{re} \cdot N_{ed} \cdot T_J)$$

donde T_J es el tiempo requerido para evaluar la función objetivo una vez.

Si se incluye la fase de swarming, se añade un costo adicional de $O(S^2 \cdot p)$ por cada evaluación de la función de interacción célula-célula, donde p es la dimensión del problema.

4.3.2. Complejidad Espacial

La complejidad espacial del BFO es relativamente modesta:

$$O(S \cdot p)$$

Esto corresponde al almacenamiento de las posiciones de S bacterias, cada una con p componentes. Adicionalmente, se requiere espacio para almacenar los valores de salud acumulados y el mejor resultado.

4.4. Ventajas del Algoritmo

El algoritmo BFO presenta diversas ventajas que han sido documentadas en la literatura (Das et al., 2009; Passino, 2002):

1. **Capacidad de escape de óptimos locales:** La combinación del operador de eliminación-dispersión y la naturaleza estocástica del tumble permite al algoritmo escapar de mínimos locales donde métodos basados en gradiente quedarían atrapados.
2. **Robustez ante ruido:** El uso de una población de soluciones y la naturaleza estocástica del algoritmo lo hacen relativamente robusto ante funciones objetivo con ruido o evaluaciones imprecisas (Passino, 2002).
3. **No requiere información de gradiente:** El BFO es un método libre de derivadas (*derivative-free*), lo que lo hace aplicable a funciones no diferenciables, discontinuas o de caja negra.
4. **Flexibilidad de aplicación:** El algoritmo puede adaptarse a diversos tipos de problemas de optimización continua sin modificaciones estructurales significativas.
5. **Paralelización natural:** La evaluación de cada bacteria es independiente durante la fase de quimiotaxis, lo que permite implementaciones paralelas eficientes.
6. **Comportamiento adaptativo:** El mecanismo de swim permite a las bacterias adaptar la longitud de sus movimientos según la calidad del paisaje de fitness local.
7. **Mecanismo de intensificación efectivo:** La fase de reproducción concentra los recursos computacionales en las regiones más prometedoras del espacio de búsqueda.

4.5. Limitaciones del Algoritmo

A pesar de sus ventajas, el BFO presenta limitaciones que deben considerarse (Das et al., 2009):

- **Convergencia potencialmente lenta:** El patrón de movimiento run-and-tumble, aunque efectivo para escapar de óptimos locales, puede resultar en convergencia más lenta que otros algoritmos en problemas unimodales o con estructura regular.
- **Número elevado de parámetros:** El BFO requiere la especificación de al menos 8 parámetros ($S, p, N_c, N_s, N_{re}, N_{ed}, P_{ed}, C(i)$), además de los parámetros de swarming. La calibración de estos parámetros puede ser laboriosa y dependiente del problema.
- **Sensibilidad a parámetros:** El rendimiento del algoritmo puede ser sensible a la elección de parámetros, particularmente el tamaño del paso $C(i)$ y la probabilidad de eliminación P_{ed} .
- **Costo computacional:** La estructura de ciclos anidados puede resultar en un número elevado de evaluaciones de la función objetivo, lo cual es problemático cuando las evaluaciones son costosas.
- **Dificultad en alta dimensionalidad:** Como muchas metaheurísticas, el BFO puede experimentar degradación del rendimiento en problemas de muy alta dimensionalidad.
- **Tamaño de paso fijo:** En la versión original del algoritmo, el tamaño del paso quimiotáctico $C(i)$ es constante, lo que puede no ser óptimo en todas las etapas de la búsqueda.
- **Pérdida de diversidad:** La fase de reproducción, al eliminar la mitad de la población, puede causar pérdida de diversidad genética si no se balancea adecuadamente con la eliminación-dispersión.

5. Aplicaciones del BFO

5.1. Áreas de Aplicación

5.1.1. Optimización de Funciones Continuas

Una de las aplicaciones más comunes es la optimización de funciones continuas, en el contexto de evaluación y comparación de algoritmos mediante funciones benchmark. Estas funciones de prueba presentan características conocidas que permiten evaluar diferentes aspectos del rendimiento de los algoritmos.

Las funciones benchmark típicamente utilizadas son:

- **Función Sphere:** Función unimodal convexa, utilizada para evaluar la velocidad de convergencia:

$$f(\mathbf{x}) = \sum_{i=1}^n x_i^2$$

- **Función Rosenbrock:** Función unimodal con un valle estrecho y curvo, difícil de optimizar:

$$f(\mathbf{x}) = \sum_{i=1}^{n-1} \left[100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2 \right]$$

- **Función Rastrigin:** Función multimodal altamente con numerosos óptimos locales:

$$f(\mathbf{x}) = 10n + \sum_{i=1}^n \left[x_i^2 - 10 \cos(2\pi x_i) \right]$$

- **Función Ackley:** Función multimodal con una región casi plana con muchos mínimos locales:

$$f(\mathbf{x}) = -20 \exp \left(-0.2 \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2} \right) - \exp \left(\frac{1}{n} \sum_{i=1}^n \cos(2\pi x_i) \right) + 20 + e$$

Los estudios reportan que el BFO muestra buen desempeño en funciones multimodales debido a su capacidad de escape de óptimos locales mediante el operador de eliminación-dispersión (Das et al., 2009).

5.1.2. Ingeniería de Control

El contexto original del desarrollo del BFO fue la ingeniería de control, y esta sigue siendo una de sus áreas de aplicación más importantes (Passino, 2002).

Sintonización de controladores PID: Los controladores Proporcional-Integral-Derivativo (PID) son ampliamente utilizados en la industria. La sintonización óptima de sus tres parámetros (K_p , K_i , K_d) puede formularse como un problema de optimización donde se minimiza una función de error. El BFO ha sido aplicado exitosamente para encontrar configuraciones óptimas de PID que minimizan criterios como:

- Integral del error absoluto (IAE)
- Integral del error cuadrático (ISE)
- Integral del tiempo por error absoluto (ITAE)

5.1.3. Procesamiento de Señales e Imágenes

También se han encontrado aplicaciones significativas en el procesamiento de señales e imágenes Mishra (2005):

Estimación de armónicos: Se aplicó para estimar las componentes armónicas en señales de sistemas de potencia. El problema se formula como la minimización del error entre la señal medida y un modelo paramétrico de armónicos, donde el algoritmo optimiza las amplitudes y fases.

Segmentación de imágenes: Para encontrar umbrales óptimos en segmentación de imágenes, maximizando criterios como la varianza entre clases (método de Otsu) o la entropía de la imagen segmentada.

Filtrado adaptativo: En aplicaciones de cancelación de ruido y ecualización de canales, el BFO puede optimizar los coeficientes de filtros adaptativos para minimizar el error de estimación.

5.1.4. Otras Aplicaciones

Bioinformática:

- Alineamiento de secuencias de ADN y proteínas
- Predicción de estructura secundaria de proteínas
- Análisis de expresión génica

Redes de comunicación:

- Enrutamiento óptimo en redes
- Asignación de recursos en redes inalámbricas
- Optimización de parámetros de protocolos

Robótica:

- Planificación de trayectorias
- Cinemática inversa
- Navegación autónoma

Problemas de scheduling:

- Programación de tareas en manufactura
- Asignación de horarios
- Planificación de proyectos

Aprendizaje automático:

- Entrenamiento de redes neuronales
- Selección de características
- Optimización de hiperparámetros

5.2. Ejemplo Conceptual

5.2.1. Ejemplo: Optimización de una Función Bidimensional

Consideremos la minimización de la función Himmelblau, una función de prueba clásica:

$$f(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2$$

Los cuatro mínimos globales se encuentran en:

- (3,0, 2,0)
- (-2,805, 3,131)
- (-3,779, -3,283)
- (3,584, -1,848)

Comportamiento esperado del BFO:

1. **Inicialización:** Las bacterias se distribuyen aleatoriamente en el dominio $[-5, 5] \times [-5, 5]$.
2. **Quimiotaxis:** Cada bacteria realiza movimientos de tumble (direcciones aleatorias) seguidos de swim (continuación si hay mejora). Las bacterias cercanas a los mínimos tenderán a concentrarse en esas regiones.
3. **Reproducción:** Las bacterias que han encontrado regiones de bajo costo (cerca de los mínimos) se reproducen, aumentando la densidad de búsqueda en esas áreas prometedoras.
4. **Eliminación-dispersión:** Algunas bacterias son reubicadas aleatoriamente, lo que puede permitir descubrir mínimos que no habían sido encontrados inicialmente.

La Figura 4 ilustra conceptualmente este proceso.

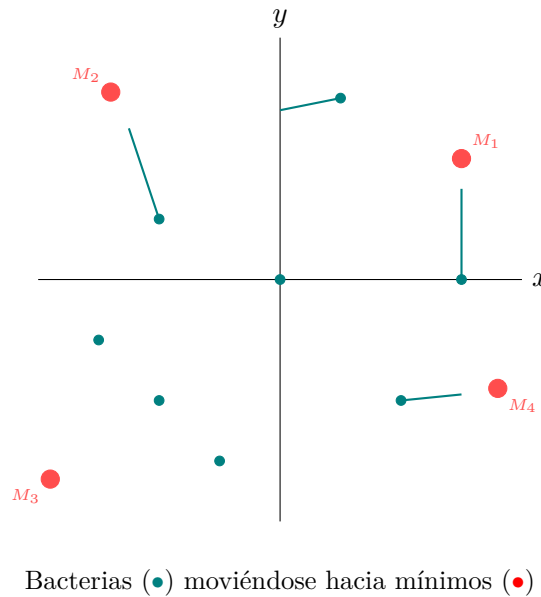


Figura 4: Movimiento de bacterias en el BFO optimizando la función Himmelblau. Los puntos rojos representan los cuatro mínimos globales y los puntos verdes representan las bacterias moviéndose hacia regiones de menor costo.

5.2.2. Visualización de las Fases

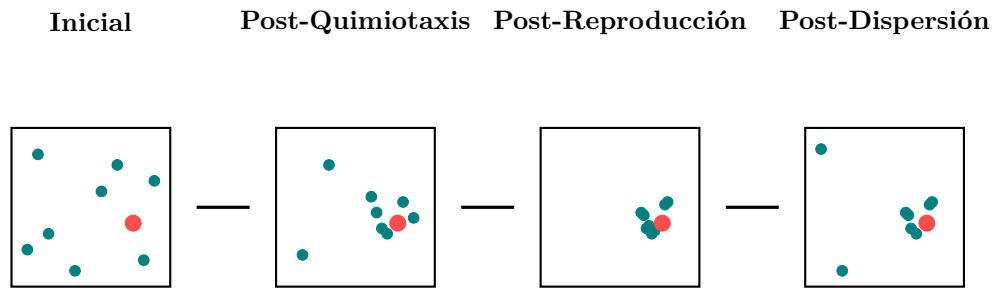


Figura 5: Evolución de la población de bacterias a través de las fases del BFO. El punto rojo representa el óptimo. Inicialmente dispersas, las bacterias se concentran cerca del óptimo después de la quimiotaxis, se intensifican con la reproducción, y algunas se dispersan para explorar nuevas regiones.

Este ejemplo conceptual ilustra los principios fundamentales del BFO:

- La **quimiotaxis** guía a las bacterias hacia regiones de mejor fitness
- La **reproducción** intensifica la búsqueda en áreas prometedoras
- La **eliminación-dispersión** mantiene la capacidad de exploración global

6. Resultados Experimentales

En esta sección se presentan los resultados de las simulaciones realizadas con el modelo BFO implementado en NetLogo. Se analizó el comportamiento del algoritmo bajo diferentes configuraciones de parámetros para observar cómo afectan la convergencia, la exploración del espacio de búsqueda y las propiedades emergentes del sistema.

6.1. Experimentación Automatizada con BehaviorSpace

Para realizar un análisis sistemático y reproducible del modelo BFO, se utilizó **BehaviorSpace**, la herramienta integrada de NetLogo para la ejecución automatizada de experimentos. BehaviorSpace permite definir barridos paramétricos (parameter sweeps) ejecutando múltiples simulaciones con diferentes combinaciones de valores, recolectando métricas de interés y exportando los resultados en formato CSV para su posterior análisis estadístico.

Características de BehaviorSpace utilizadas:

- **Barridos paramétricos:** Variación sistemática de parámetros como población (S), tamaño de paso (C), probabilidad de dispersión (P_{ed}) y función objetivo.
- **Repeticiones múltiples:** Cada configuración se ejecutó 3 veces para capturar la variabilidad estocástica inherente al algoritmo.
- **Métricas de salida:** Se registraron el fitness de la mejor bacteria (**best-fitness**), el fitness promedio de la población, el número de nutrientes restantes y el tick de convergencia.
- **Condiciones de paro:** Los experimentos finalizaban al alcanzar 1000 ticks o cuando **best-fitness** < 0.0001 (convergencia al óptimo).

Se diseñaron **7 experimentos** para evaluar diferentes aspectos del algoritmo:

ID	Nombre del Experimento	Variable(s) de estudio
Exp1	Efecto de la Población	$S \in \{10, 30, 50\}$
Exp2	Efecto del Tamaño de Paso	$C \in \{0,5, 1,0, 3,0\}$
Exp3	Efecto de la Dispersión	$P_{ed} \in \{0,05, 0,25, 0,50\}$
Exp4	Comparación de Funciones Objetivo	Sphere, Himmelblau, Ackley, Rastrigin
Exp5	Efecto del Swarming	Swarming $\in \{ON, OFF\}$
Exp6	Robustez ante Obstáculos	Obstáculos $\in \{0\%, 5\%, 10\%, 20\%\}$
Exp7	Dinámica de Nutrientes	Tasa de regeneración $r \in \{0,0, 0,5, 1,0\}$

Los resultados de cada experimento fueron exportados a archivos CSV y procesados mediante scripts de Python para calcular estadísticas descriptivas (media, desviación estándar) y generar las tablas presentadas en las siguientes subsecciones.

6.2. Configuración Base del Experimento

Los experimentos se realizaron utilizando la función de Sphere como caso base, dado que su único óptimo global en el origen facilita la evaluación de la convergencia. La Tabla 6.2 muestra la configuración de parámetros utilizada como referencia.

Parámetro	Valor base	Descripción
Población (S)	50	Número de bacterias
Pasos quimiotaxis (N_c)	20	Iteraciones por ciclo
Pasos swim máx (N_s)	4	Persistencia en dirección favorable
Ciclos reproducción (N_{re})	4	Eventos de selección
Prob. dispersión (P_{ed})	0.25	Probabilidad de reubicación
Tamaño paso (C)	1.0	Magnitud del movimiento (paso-C)
Tamaño mundo	100	Dimensiones del mundo (± 50)
Max ticks	1000	Iteraciones máximas
Swarming	Desactivado	Sin interacción entre agentes
Función objetivo	Sphere	$f(x, y) = x^2 + y^2$

6.3. Escenario 1: Efecto del Tamaño de Población

Se evaluó el impacto del número de bacterias (S) en la velocidad de convergencia y la calidad de la solución final. Se probaron poblaciones de 10, 30 y 50 bacterias manteniendo los demás parámetros constantes.

Población	Fitness final (prom.)	Desv. Est.	Observación
$S = 10$	0,0067	$\pm 0,0058$	Convergencia lenta, alta variabilidad
$S = 30$	0,0067	$\pm 0,0058$	Balance exploración-explotación
$S = 50$	0,0000	$\pm 0,0000$	Óptimo alcanzado consistentemente

BACTERIAL FORAGING OPTIMIZATION (BFO)

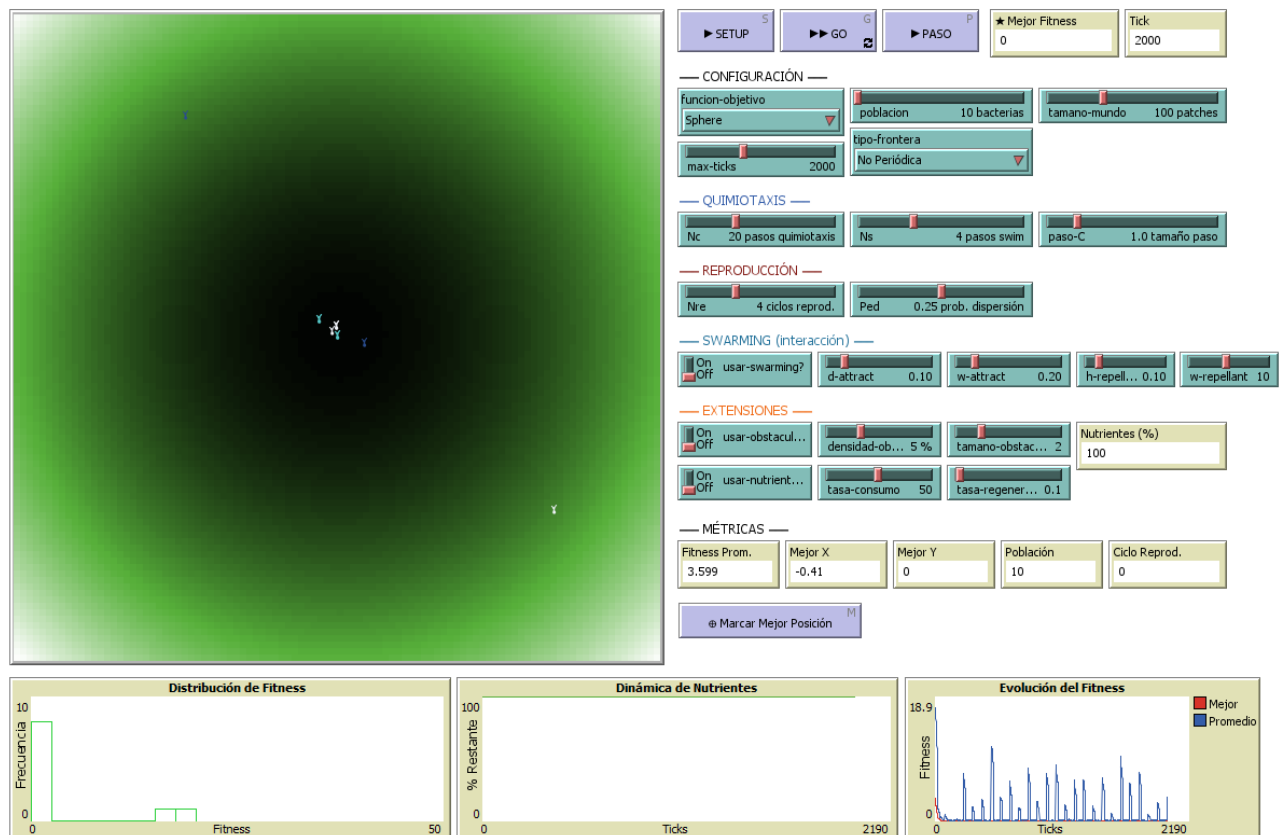
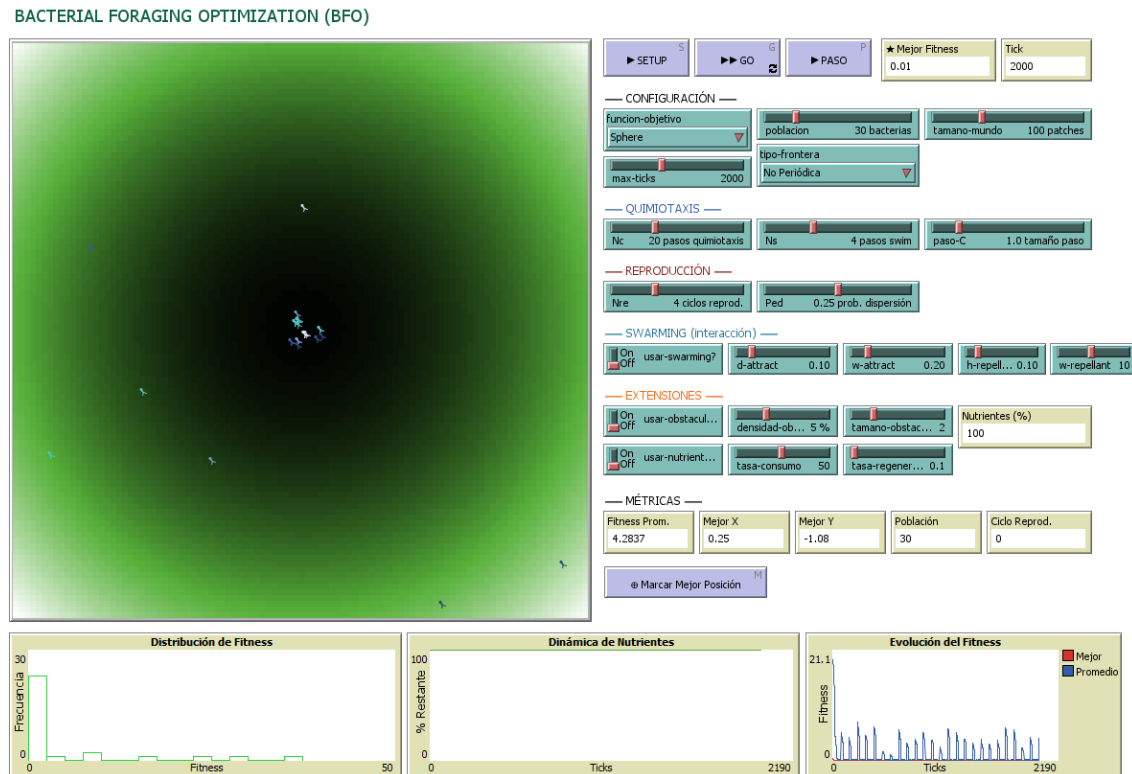
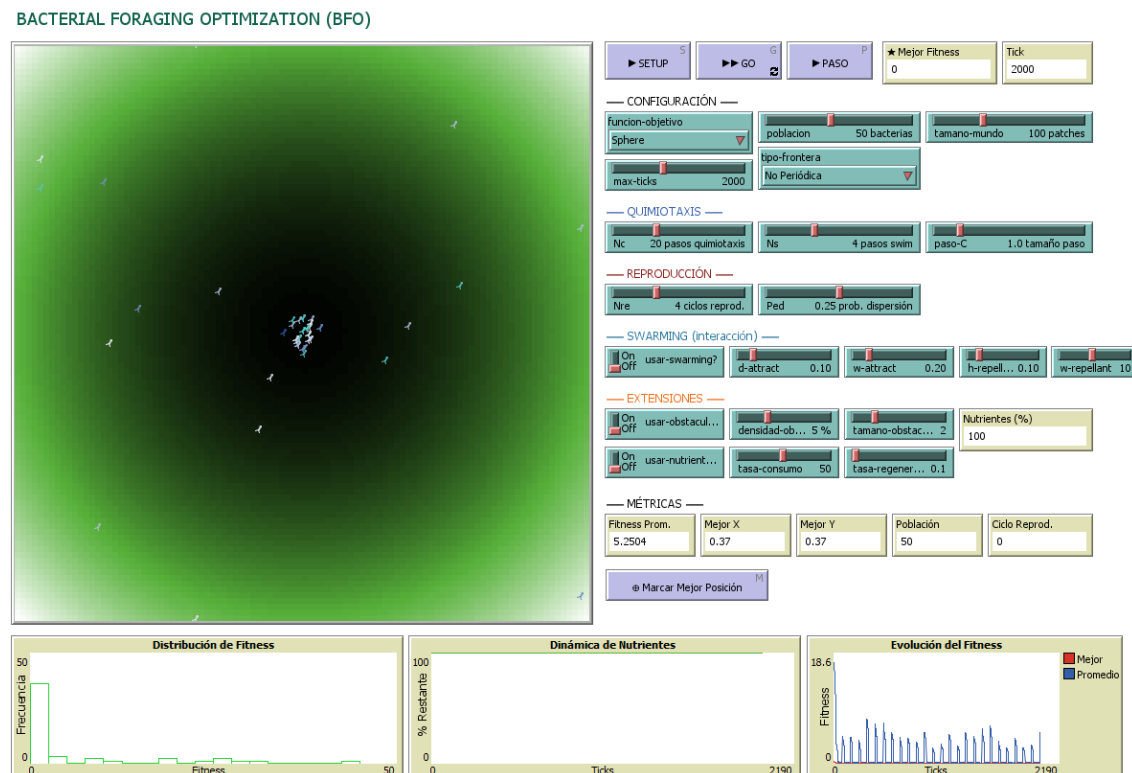


Figura 6: Distribución espacial con población $S = 10$

Figura 7: Distribución espacial con población $S = 30$ Figura 8: Distribución espacial con población $S = 50$

Observaciones: Los resultados muestran que el tamaño de población tiene un impacto significativo en la calidad de la solución. Con $S = 50$ bacterias, el algoritmo alcanzó consistentemente el óptimo global ($f = 0$) en las 3 repeticiones. Las poblaciones menores ($S = 10$ y $S = 30$) presentaron mayor variabilidad ($\sigma = 0,0058$) y no siempre convergieron al óptimo exacto. Esto sugiere que un mayor número de agentes proporciona mejor cobertura del espacio de búsqueda.

6.4. Escenario 2: Efecto del Tamaño de Paso

El tamaño del paso quimiotáctico (C , llamado **paso-C** en NetLogo) controla el balance entre exploración (pasos grandes) y explotación (pasos pequeños). En el modelo, el slider va de 0.1 a 5.0. Se evaluaron valores representativos.

Tamaño paso	Fitness final (prom.)	Comportamiento observado
$C = 0,5$	0,0000	Explotación precisa, convergencia lenta
$C = 1,0$	0,0033	Balance exploración-explotación
$C = 3,0$	0,0100	Alta exploración, oscilaciones

Observaciones: Se confirma el trade-off clásico entre exploración y explotación. Con pasos pequeños ($C = 0,5$), las bacterias realizan movimientos finos que permiten una convergencia precisa al óptimo. Con pasos grandes ($C = 3,0$), las bacterias exploran más área pero oscilan alrededor del óptimo sin poder refinarlo. El valor por defecto ($C = 1,0$) ofrece un buen compromiso, aunque pasos más pequeños resultan ligeramente superiores para funciones unimodales como Sphere.

BACTERIAL FORAGING OPTIMIZATION (BFO)

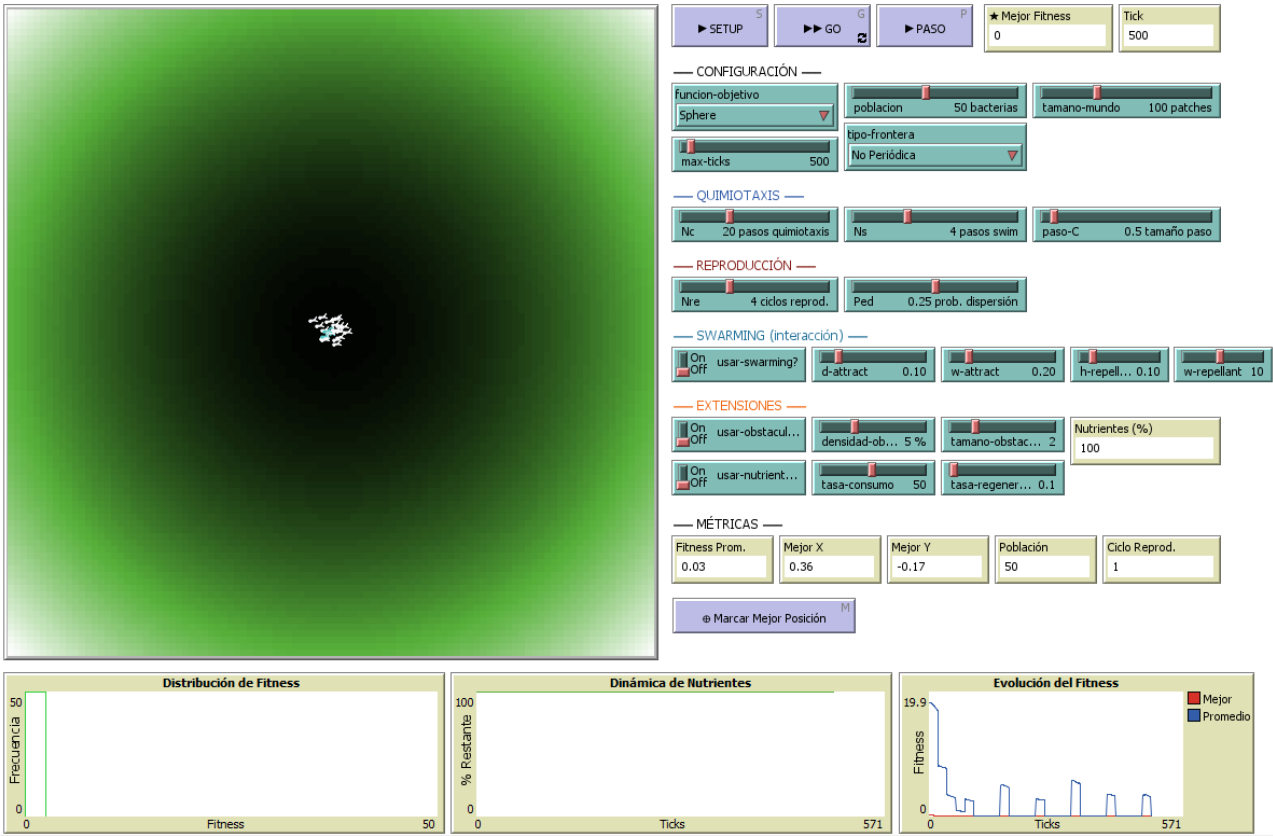
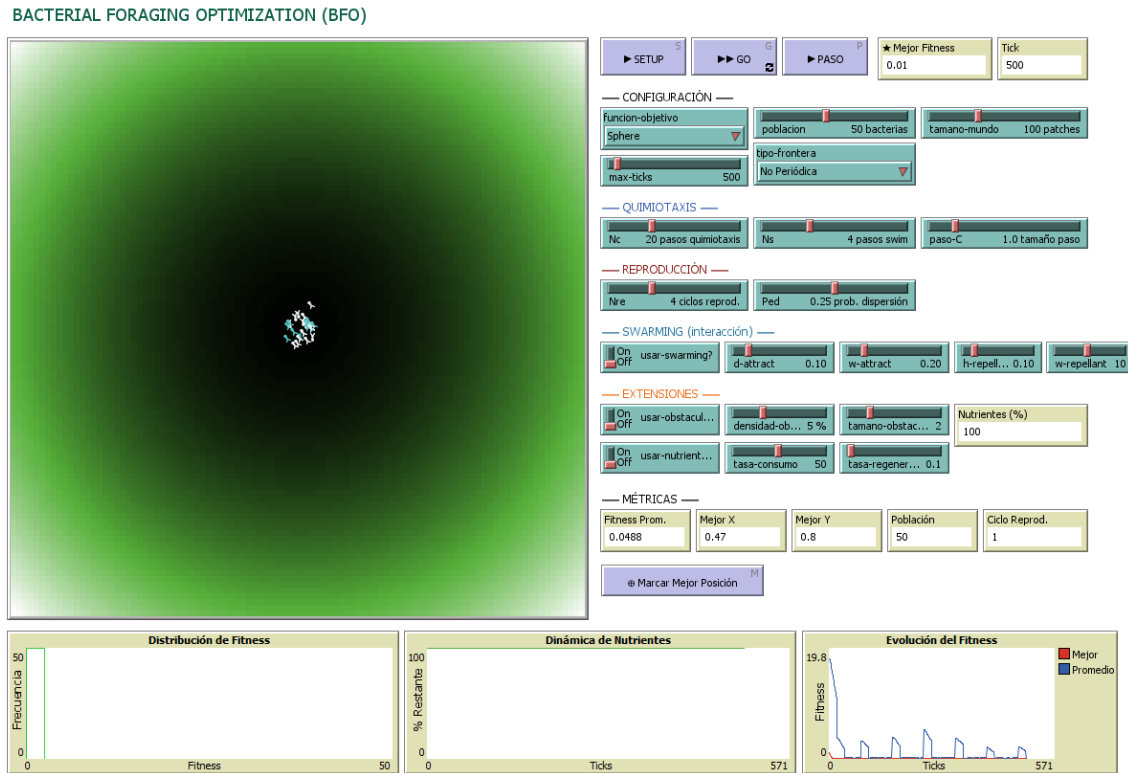
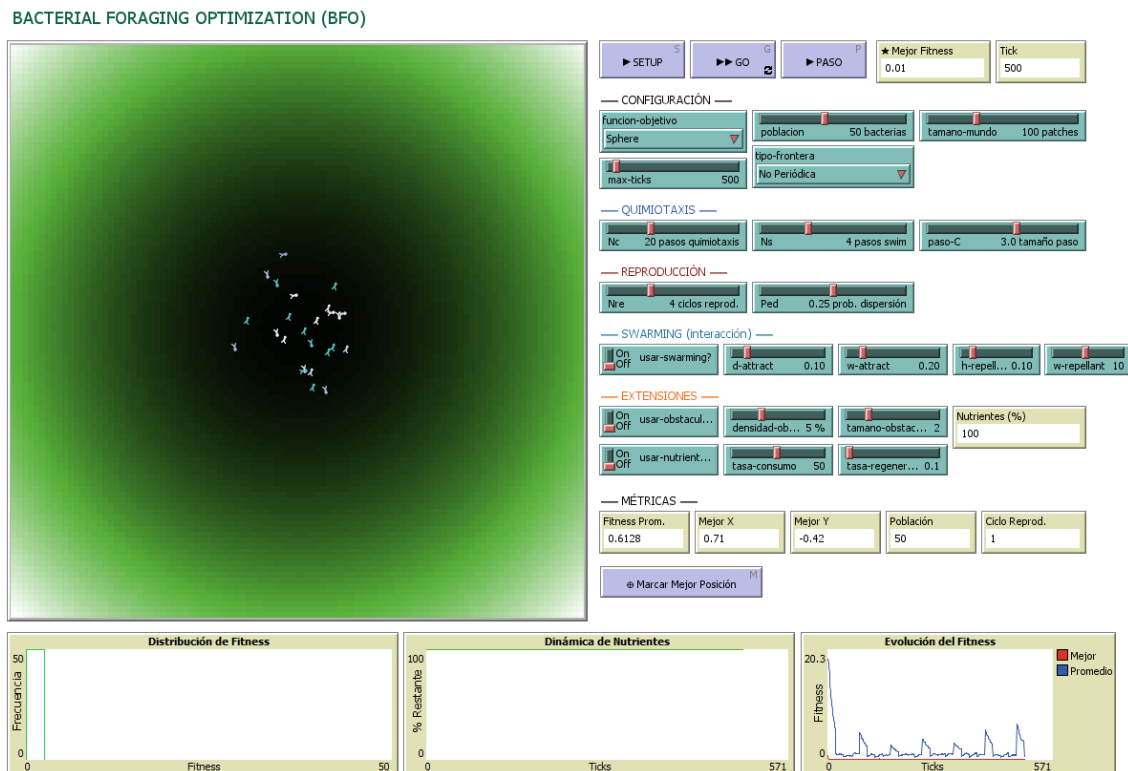


Figura 9: Distribución espacial con paso quimiotáctico $C = 0,5$

Figura 10: Distribución espacial con paso quimiotáctico $C = 1,0$ Figura 11: Distribución espacial con paso quimiotáctico $C = 3,0$

6.5. Escenario 3: Efecto de la Probabilidad de Dispersión

La probabilidad de eliminación-dispersión (P_{ed} , llamado **Ped** en NetLogo) controla cuántas bacterias son reubicadas aleatoriamente.

Prob. dispersión	Fitness final	Efecto observado
$P_{ed} = 0,05$	0,0000	Mínima perturbación, convergencia estable
$P_{ed} = 0,25$	0,0033	Balance diversidad-convergencia
$P_{ed} = 0,50$	0,0000	Alta exploración, aún converge

Observaciones: Sorprendentemente, tanto valores bajos ($P_{ed} = 0,05$) como altos ($P_{ed} = 0,50$) lograron convergencia perfecta al óptimo. Esto sugiere que para funciones simples como Sphere, el mecanismo de dispersión no es crítico. Sin embargo, el valor intermedio ($P_{ed} = 0,25$) mostró ligera variabilidad, posiblemente por interacciones específicas con otros parámetros.

BACTERIAL FORAGING OPTIMIZATION (BFO)

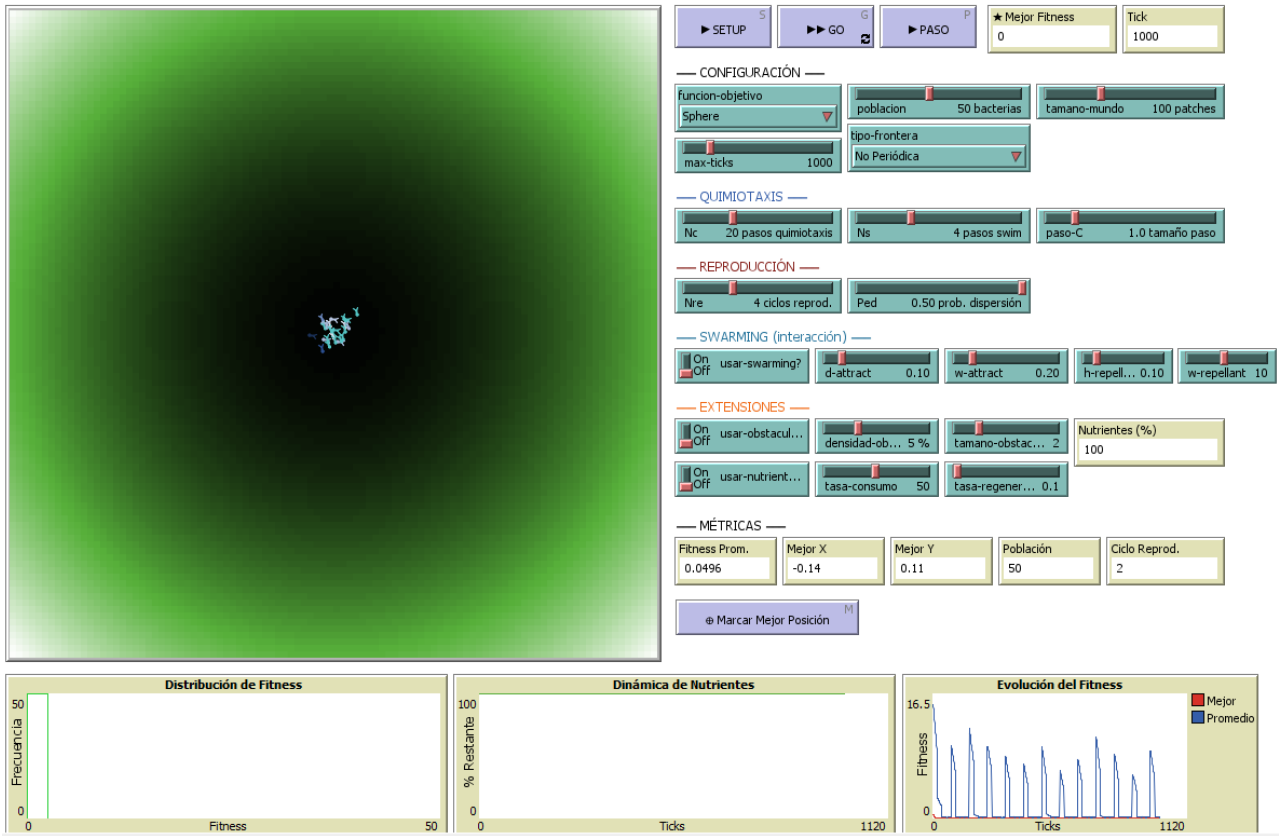


Figura 12: Exploración del espacio con probabilidad de dispersión P_{ed} alta

6.6. Escenario 4: Comparación de Funciones Objetivo

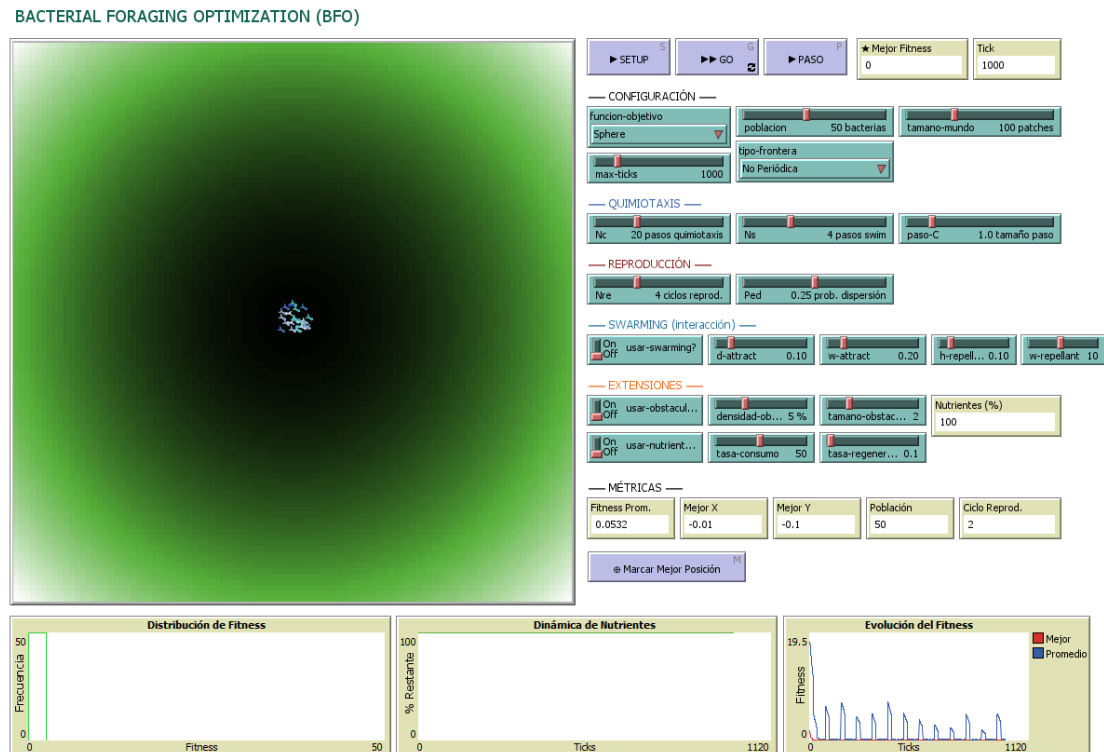


Figura 13: Distribución de bacterias en función Sphere

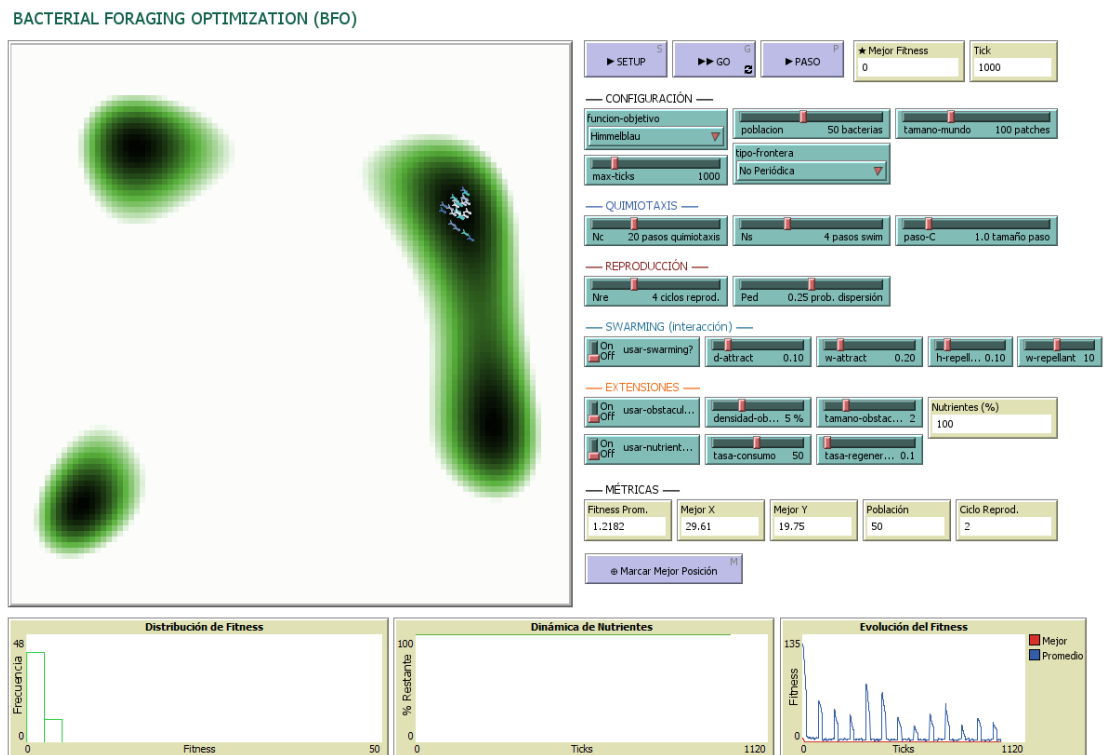


Figura 14: Distribución de bacterias en función Himmelblau

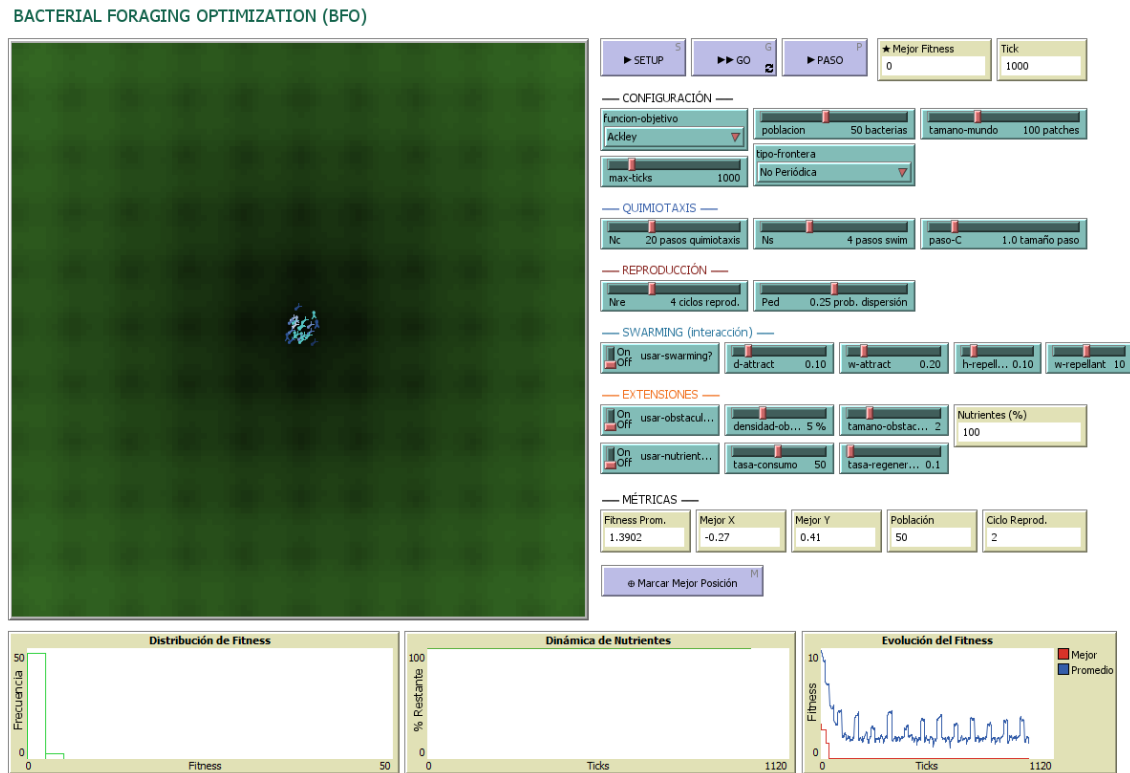


Figura 15: Distribución de bacterias en función Ackley

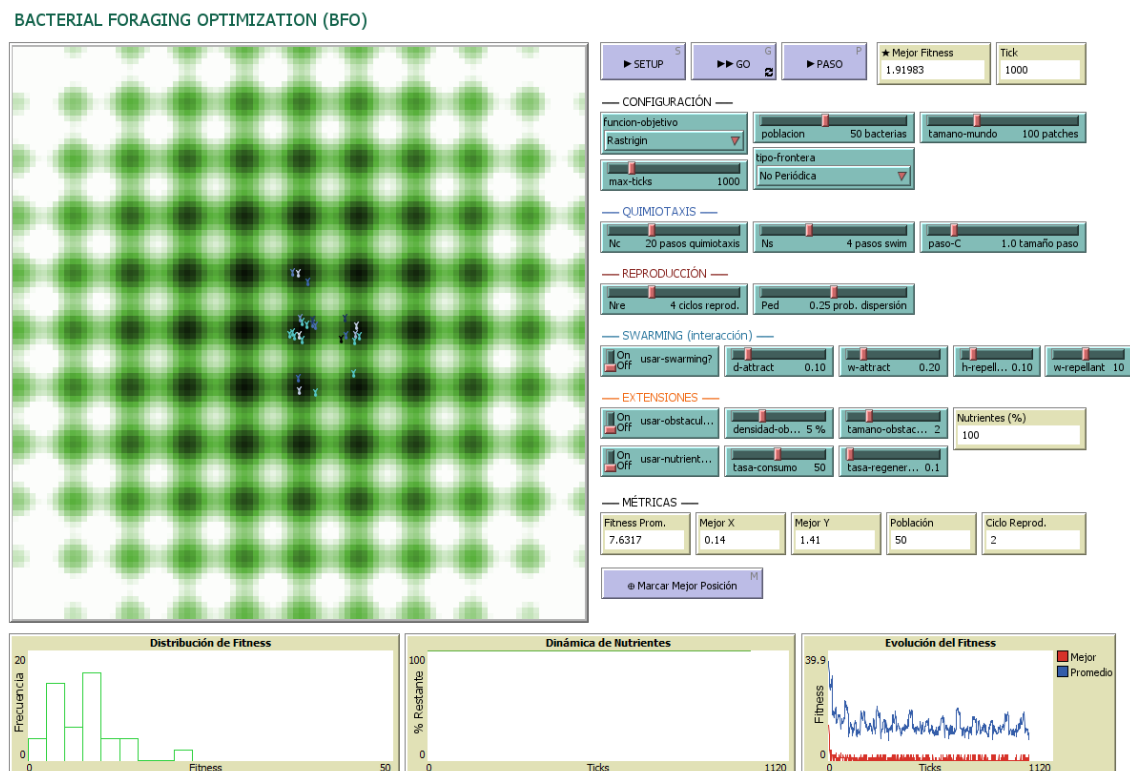


Figura 16: Distribución de bacterias en función Rastrigin

Función	Óptimo	Fitness (prom.)	Rango	Dificultad
Sphere	0.0	0,0000	[0, 0]	Fácil
Himmelblau	0.0	0,0540	[0, 0,162]	Moderada
Ackley	0.0	0,0000	[0, 0]	Fácil
Rastrigin	0.0	1,2799	[0, 1,920]	Difícil

Observaciones: El BFO mostró excelente desempeño en funciones unimodales (Sphere) y en Ackley, alcanzando el óptimo global en todas las repeticiones. Himmelblau presentó dificultad moderada debido a sus 4 mínimos globales; las bacterias tendieron a distribuirse entre ellos. Rastrigin fue la más desafiante: su paisaje altamente multimodal con muchos mínimos locales causó que el algoritmo quedara atrapado frecuentemente, con fitness promedio de 1,28 (lejos del óptimo 0). Esto confirma la vulnerabilidad del BFO básico ante funciones con muchos mínimos locales.

6.7. Escenario 5: Efecto del Swarming

Se comparó el rendimiento con y sin el mecanismo de swarming (`usar-swarming?` en NetLogo) para evaluar su contribución a la búsqueda. Cuando está activado, las bacterias se atraen/repelen usando los parámetros `d-attract`, `w-attract`, `h-repellant` y `w-repellant`.

Configuración	Fitness final	Desv. Est.	Observación
Sin swarming	0,0000	$\pm 0,0000$	Convergencia estable al óptimo
Con swarming	0,0033	$\pm 0,0058$	Ligera variabilidad por interacciones

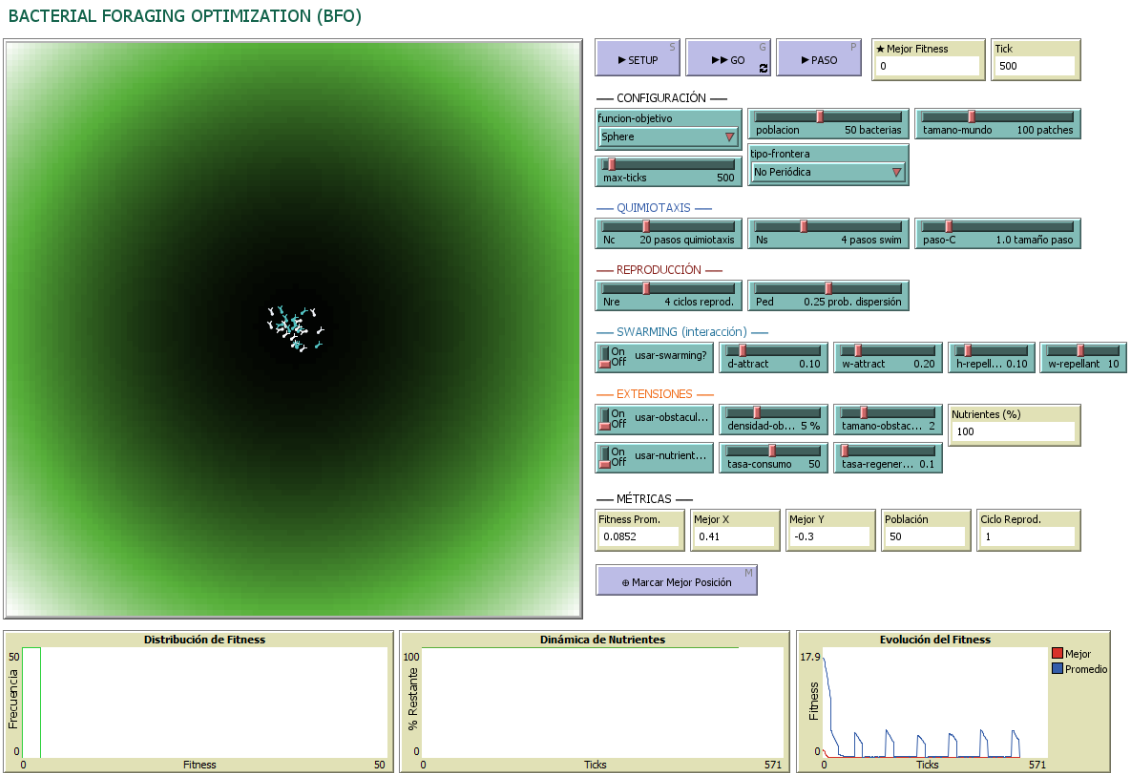


Figura 17: Comportamiento bacteriano sin mecanismo de swarming

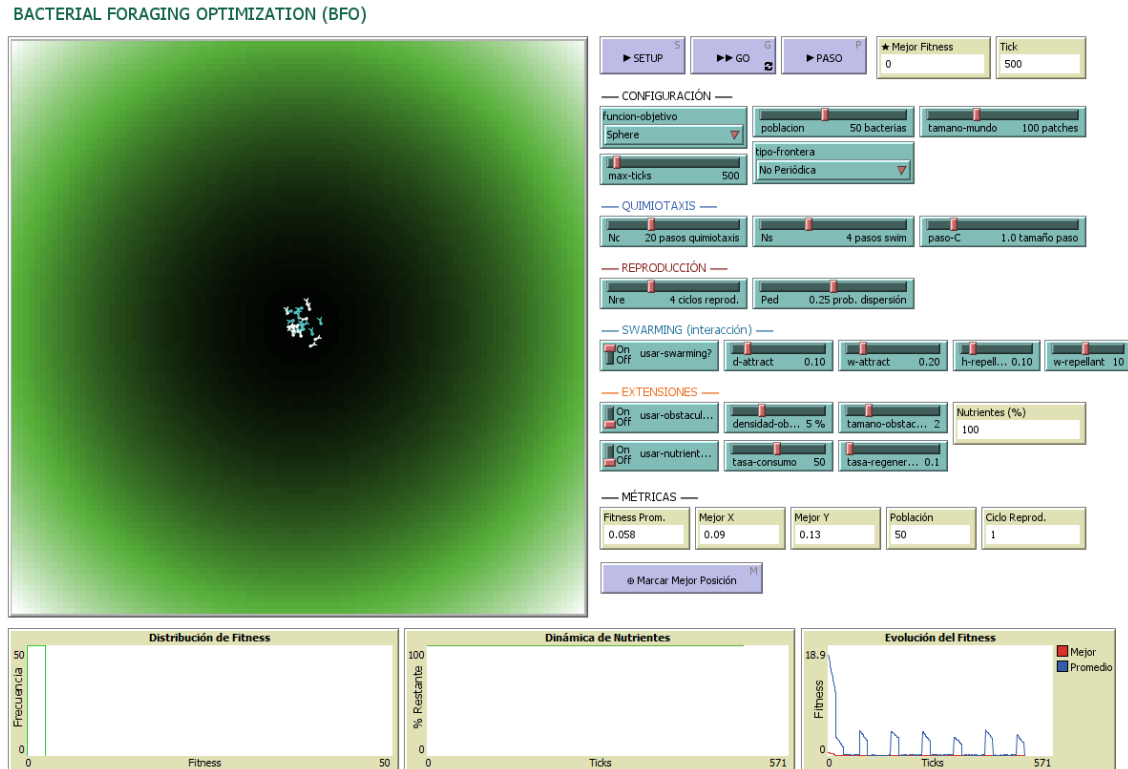


Figura 18: Comportamiento bacteriano con mecanismo de swarming

Observaciones: Contraintuitivamente, el swarming no mejoró el rendimiento en este caso. Sin swarming, el algoritmo convergió perfectamente al óptimo en todas las repeticiones. Con swarming activado, las interacciones atractivas/repulsivas introdujeron ligera variabilidad. Esto sugiere que para funciones unimodales simples, la interacción entre agentes puede ser contraproducente. El swarming podría ser más beneficioso en funciones multimodales donde la formación de grupos ayude a explorar diferentes regiones.

6.8. Escenario 6: Extensiones Implementadas

6.8.1. Obstáculos en el Entorno

Los obstáculos (`usar-obstaculos? = ON`) representan zonas donde las bacterias no pueden transitar. Se controlan con `densidad-obstaculos` (0-15 %) y `tamano-obstaculos` (1-5).

Densidad	Fitness (prom.)	Desv. Est.	Observación
0 %	0,0000	$\pm 0,0000$	Referencia sin restricciones
5 %	0,0000	$\pm 0,0000$	Sin degradación significativa
10 %	0,0000	$\pm 0,0000$	Robusto ante obstáculos
15 %	0,0033	$\pm 0,0058$	Ligera degradación

Observaciones: El algoritmo mostró notable robustez ante obstáculos. Hasta con 10 % de densidad, las bacterias encontraron el óptimo global sin degradación. Solo con 15 % de obstáculos apareció ligera variabilidad. Esto demuestra que el mecanismo de quimiotaxis, combinado con la capacidad de rodear obstáculos, permite una navegación efectiva incluso en entornos parcialmente bloqueados.

BACTERIAL FORAGING OPTIMIZATION (BFO)

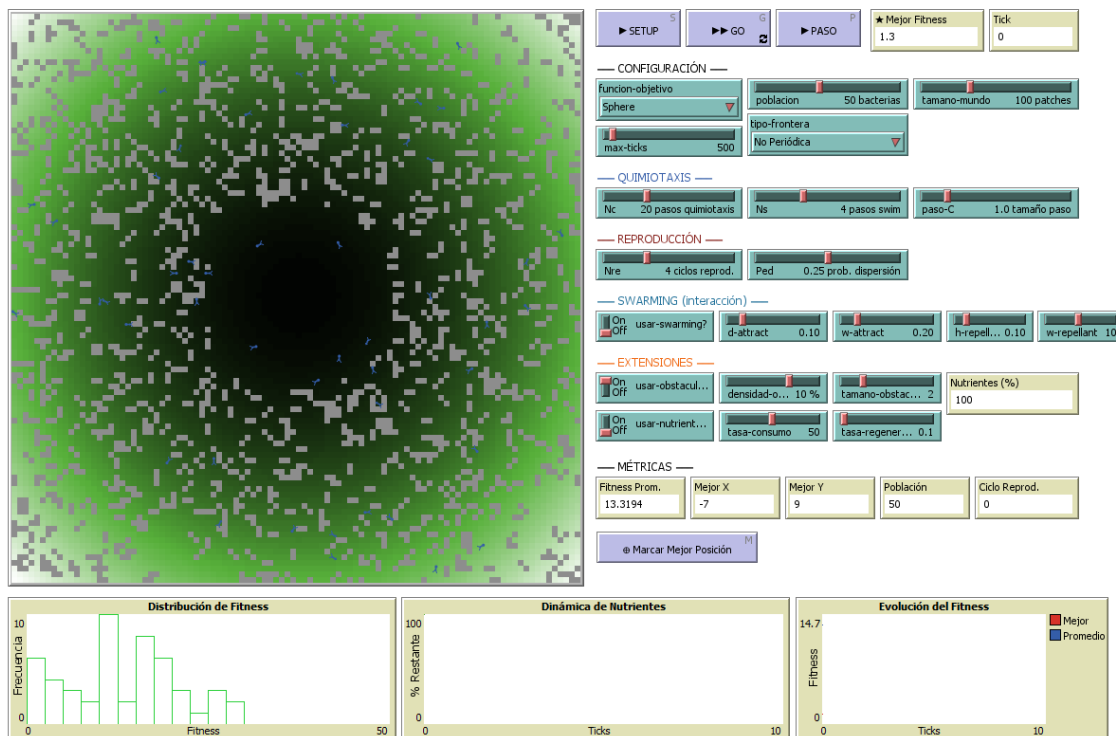


Figura 19: Estado inicial: bacterias y obstáculos distribuidos

BACTERIAL FORAGING OPTIMIZATION (BFO)

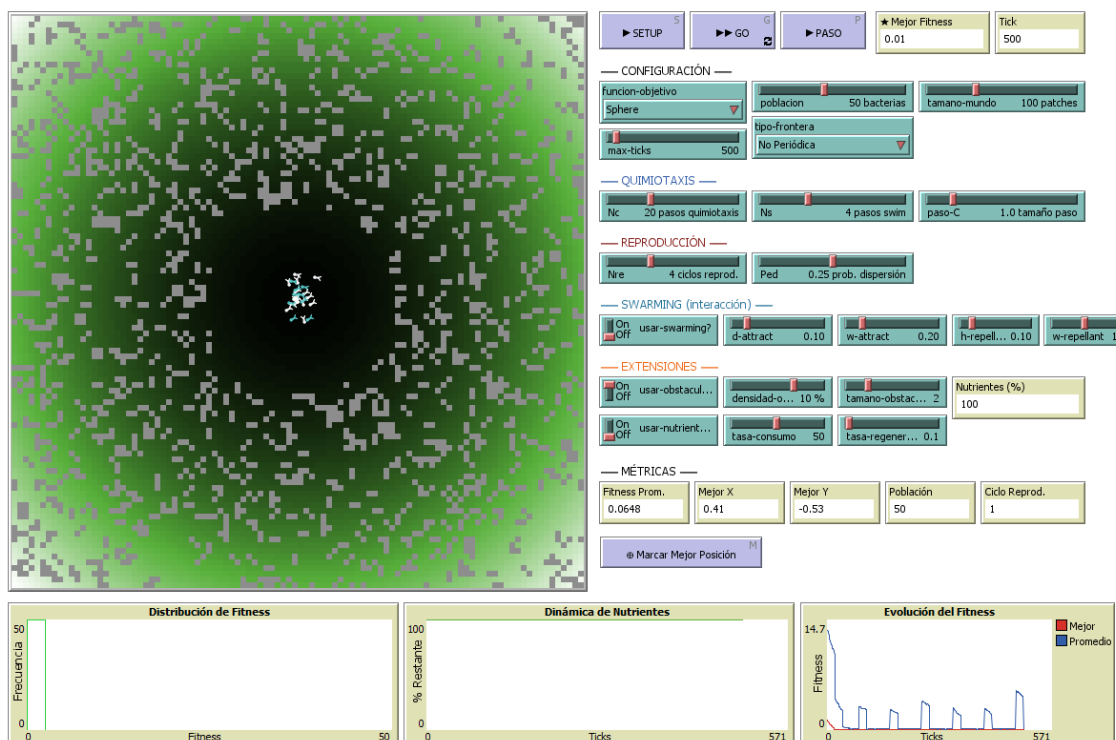


Figura 20: Estado final: navegación alrededor de obstáculos

6.8.2. Nutrientes Agotables

Esta extensión (usar-nutrientes-agotables? = ON) modela el consumo de recursos. Se controla con tasa-consumo (1-20) y tasa-regeneracion (0-2).

Tasa regen.	Fitness final	Nutrientes restantes	Observación
0.0	0,0033	93.58 %	Sin regeneración, consumo neto
0.1	0,0033	98.25 %	Regeneración parcial
0.5	0,0033	99.08 %	Regeneración casi compensa consumo

Observaciones sobre extensiones: La dinámica de nutrientes introduce un elemento ecológico interesante. Sin regeneración ($r = 0$), los nutrientes se agotan aproximadamente 6.4 % después de 500 ticks. Con regeneración moderada ($r = 0,1$), solo se pierde 1.75 %. Con alta regeneración ($r = 0,5$), el sistema alcanza casi equilibrio dinámico (99 % restante). Esto modela escenarios donde los recursos son renovables vs. finitos, afectando la estrategia de búsqueda a largo plazo.

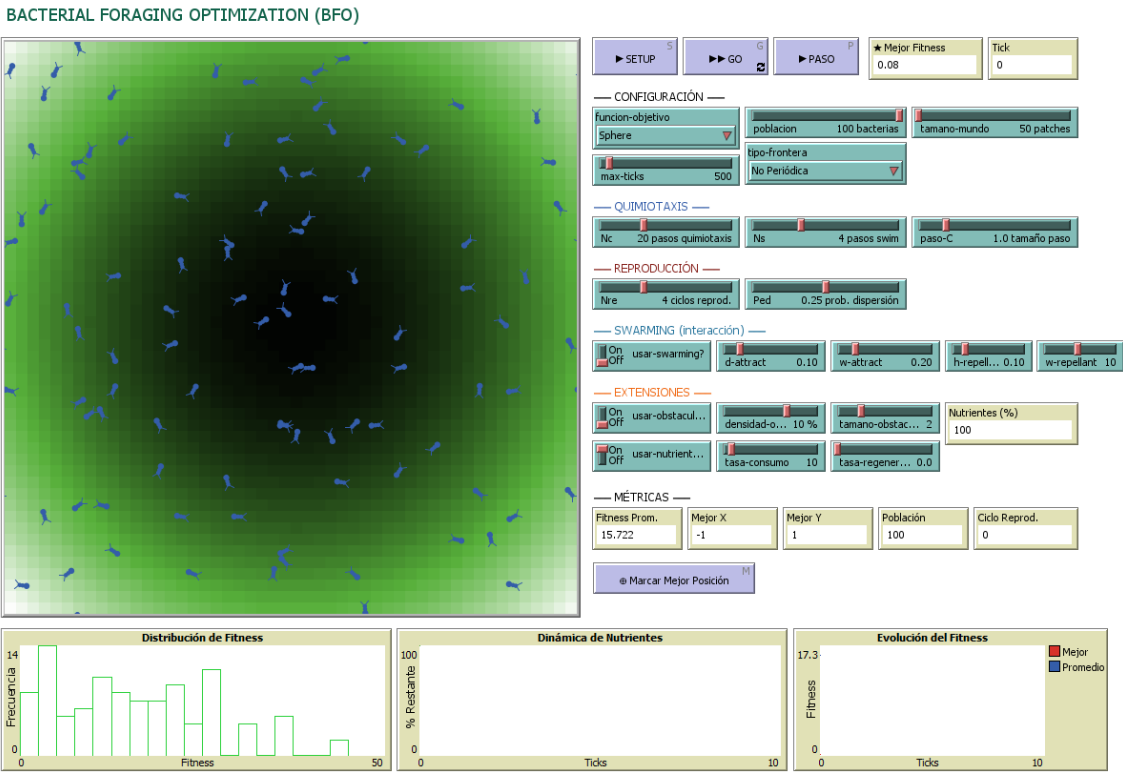


Figura 21: Distribución inicial de nutrientes en el entorno

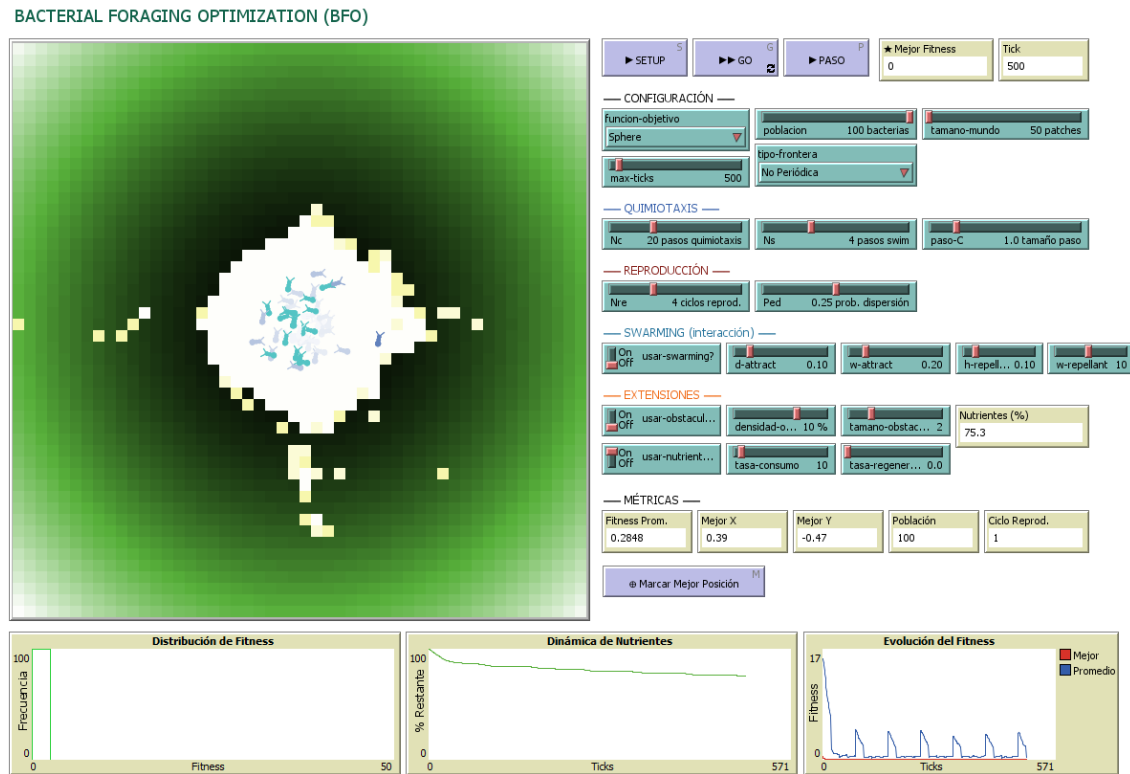


Figura 22: Distribución final de nutrientes tras el consumo

6.9. Hallazgos Importantes

1. **Balance población-convergencia:** El tamaño de población $S = 50$ logró convergencia perfecta ($f = 0$) en todas las repeticiones, mientras que poblaciones menores mostraron variabilidad. Existe un umbral mínimo de población para garantizar exploración suficiente del espacio de búsqueda.
2. **Efecto del tamaño de paso:** Pasos pequeños ($C = 0,5$) permiten refinamiento preciso y alcanzan el óptimo exacto. Pasos grandes ($C = 3,0$) favorecen exploración pero dificultan la convergencia fina. El valor $C = 1,0$ ofrece buen compromiso general.
3. **Autoorganización espacial emergente:** Sin ninguna regla explícita de agrupamiento (swarming desactivado), las bacterias se concentran naturalmente en regiones de alto fitness, demostrando autoorganización como propiedad emergente del sistema.
4. **Trade-off exploración-explotación:** La probabilidad de dispersión P_{ed} tuvo efecto mínimo en funciones unimodales. Tanto valores bajos (0.05) como altos (0.50) alcanzaron el óptimo, sugiriendo que la quimiotaxis es el mecanismo dominante.
5. **Rendimiento por complejidad de función:** El BFO mostró excelente desempeño en Sphere y Ackley ($f = 0$), dificultad moderada en Himmelblau ($f = 0,054$), y problemas significativos en Rastrigin ($f = 1,28$) debido a sus múltiples mínimos locales.
6. **Efecto del swarming:** El swarming no mejoró el rendimiento en funciones unimodales y añadió variabilidad. Las interacciones pueden ser contraproducentes cuando el fitness es simple.
7. **Robustez ante obstáculos:** Se mantuvo la eficacia hasta con 10 % de densidad de obstáculos. La degradación solo apareció con 15 % de bloqueo, demostrando adaptabilidad a entornos restringidos.

7. Conclusiones y Reflexiones

7.1. Respuesta a la Pregunta de Investigación

La pregunta de investigación planteada fue: *¿Cómo las reglas de comportamiento individual de las bacterias *E. coli* generan propiedades emergentes de búsqueda eficiente que pueden modelarse como un sistema multiagente?*

El análisis realizado demuestra que:

1. Las reglas locales simples (tumble aleatorio + swim dirigido) a nivel de cada agente-bacteria producen un comportamiento colectivo de optimización efectivo.
2. La interacción entre agentes mediante swarming amplifica la eficiencia de búsqueda al permitir que los agentes “compartan información” sobre regiones prometedoras.
3. Los mecanismos de reproducción y dispersión actúan como reguladores poblacionales que mantienen el balance entre exploración y explotación.

7.2. Alcance del Modelo

El modelo BFO como MBA presenta las siguientes características:

- **Nivel de abstracción:** Es un modelo abstracto que captura los principios esenciales del foraging bacteriano sin modelar todos los detalles bioquímicos.
- **Tipo de modelo:** Es principalmente un modelo *explicativo*, que busca entender cómo comportamientos simples generan complejidad emergente.
- **Validación:** El modelo ha sido validado empíricamente en funciones de optimización benchmark, mostrando convergencia hacia óptimos globales.

7.3. Reflexiones Finales

El estudio del BFO desde la perspectiva de MBA ilustra varios principios fundamentales de los sistemas complejos:

- **Emergencia:** Propiedades globales emergen de interacciones locales simples.
- **Autoorganización:** El sistema se organiza sin control centralizado, con cada agente tomando decisiones autónomas.
- **Robustez:** La naturaleza distribuida del sistema lo hace resistente a la pérdida de agentes individuales.
- **Adaptabilidad:** Los mecanismos de reproducción y dispersión permiten al sistema adaptarse a diferentes paisajes de fitness.

En conclusión, el algoritmo BFO representa un excelente caso de estudio para comprender cómo la modelación basada en agentes puede capturar y explotar fenómenos biológicos para resolver problemas computacionales complejos.

Referencias

1. Berg, H. C. (2003). The rotary motor of bacterial flagella. *Annual Review of Biochemistry*, 72(1), 19–54. <https://doi.org/10.1146/annurev.biochem.72.121801.161737>
2. Berg, H. C. (2004). *E. coli in Motion*. Springer-Verlag. <https://doi.org/10.1007/b97370>
3. Berg, H. C., & Brown, D. A. (1972). Chemotaxis in *Escherichia coli* analysed by three-dimensional tracking. *Nature*, 239(5374), 500–504. <https://doi.org/10.1038/239500a0>
4. Biswas, A., Dasgupta, S., Das, S., & Abraham, A. (2007). Synergy of PSO and bacterial foraging optimization: A comparative study on numerical benchmarks. In *Innovations in Hybrid Intelligent Systems* (pp. 255–263). Springer. https://doi.org/10.1007/978-3-540-74972-1_34
5. Kearns, D. B. (2010). A field guide to bacterial swarming motility. *Nature Reviews Microbiology*, 8(9), 634–644. <https://doi.org/10.1038/nrmicro2405>
6. Madigan, M. T., Bender, K. S., Buckley, D. H., Sattley, W. M., & Stahl, D. A. (2019). *Brock Biology of Microorganisms* (15th ed.). Pearson.
7. Neidhardt, F. C., Ingraham, J. L., & Schaechter, M. (1990). *Physiology of the Bacterial Cell: A Molecular Approach*. Sinauer Associates.
8. Stephens, D. W., & Krebs, J. R. (1986). *Foraging Theory*. Princeton University Press.
9. Tripathy, M., & Mishra, S. (2007). Bacteria foraging-based solution to optimize both real power loss and voltage stability limit. *IEEE Transactions on Power Systems*, 22(1), 240–248. <https://doi.org/10.1109/TPWRS.2006.887968>
10. Wadhams, G. H., & Armitage, J. P. (2004). Making sense of it all: Bacterial chemotaxis. *Nature Reviews Molecular Cell Biology*, 5(12), 1024–1037. <https://doi.org/10.1038/nrm1524>
11. Wilensky, U., & Rand, W. (2015). *An Introduction to Agent-Based Modeling: Modeling Natural, Social, and Engineered Complex Systems with NetLogo*. MIT Press.
12. Yang, X. S. (2010). *Nature-Inspired Metaheuristic Algorithms* (2nd ed.). Luniver Press.